
Documentação de Projeto

para o sistema

Gerenciamento de Inadimplentes

Versão 1.6

Projeto de sistema elaborado pelo aluno Matheus Pereira e apresentado ao curso de **Engenharia de Software** da **PUC Minas** como parte do Trabalho de Conclusão de Curso (TCC) sob orientação de conteúdo do professor Cleiton Silva Tavares, Danilo de Quadros Mais Filho , Leonardo Vilela Cardoso e Raphael Ramos Dias Costas, orientação acadêmica do professor Cleiton e orientação de TCC II do professor Cleiton.

14/06/2026

Tabela de Conteúdo

1. Introdução	3
2. Modelo de Usuário e Requisitos	3
2.1 Descrição de Atores	4
2.2 Modelo de Casos de Uso	5
2.3 Modelo de Casos de Uso e Histórias de Usuários	7
2.4 Diagrama de Sequência do Sistema e Contrato de Operações	12
3. Modelos de Projetos	13
3.1 Diagrama de Classe	14
3.2 Diagrama de Sequência	21
3.3 Diagramas de Comunicação	24
3.4 Arquitetura	26
3.5 Diagrama de Estado	30
3.6 Diagrama de Componentes e Implantação	33
4. Projeto de Interface do Usuário	42
5. Glossário e Modelos de Dados	45
6. Casos de Testes	58
7. Cronograma	59
8. Processos de Implantação	60
9. Post-mortem	62
10. Trabalhos Futuros	63

Histórico de Revisões

Nome	Data	Razões para Mudança	Versão
Matheus Pereira	28/09/25	Preenchimento das seções 1 até 2.1	1.0
Matheus Pereira	09/10/25	Preenchimento das seções 2.1,2,2,2.3,3.1, 3.2, 3.3	1.1
Matheus Pereira	02/11/25	Preenchimento das seções 3.4,3.5,3.6 e 5	1.2
Matheus Pereira	18/11/25	Preenchimento das seções 6 e 7	1.3
Matheus Pereira	30/11/25	Atualização das seções 6 e 7	1.4
Matheus Pereira	14/12/25	Correções pos banca	1.5
Matheus Pereira	14/06/26	Alterações para a Banca de TCC 2	1.6

1. Introdução

O presente documento contempla a elaboração e a revisão de modelos de domínio, bem como a definição de modelos de projeto referentes ao Sistema de Gerenciamento de Inadimplentes. Trata-se de uma aplicação voltada ao controle e acompanhamento de clientes inadimplentes, permitindo o registro de débitos, o monitoramento de pagamentos em atraso e a emissão de relatórios gerenciais que apoiam a tomada de decisão. Tais modelos têm como objetivo fornecer uma representação estruturada dos principais elementos do sistema, suas interações e responsabilidades, subsidiando, assim, o processo de desenvolvimento da aplicação de maneira consistente e alinhada aos requisitos previamente levantados.

A motivação para o desenvolvimento deste trabalho surgiu a partir de uma demanda real do escritório, de uma busca para uma solução dedicada ao controle de inadimplência. Por se tratar de um sistema interno, podendo ser utilizado por todos os funcionários do escritório. O caráter inovador está na centralização e padronização do processo de cobrança e acompanhamento, reduzindo controles manuais e oferecendo relatórios para apoiar decisões. A escolha do setor contábil se justifica pelo impacto direto da inadimplência no fluxo de caixa e na rotina operacional do escritório.

2. Modelos de Usuário e Requisitos

Esta seção tem como objetivo descrever os usuários e atores do sistema, assim como os requisitos aos quais esse deve atender. Para isso, é apresentada uma breve descrição de cada ator, assim como um modelo de caso de uso e as histórias de usuários relacionadas, ambos que servem de referência para o desenvolvimento do sistema. Por último, são apresentados o diagrama de sequência do sistema.

2.1 Descrição de Atores

Nesta subseção, são apresentados os atores que interagem com o Sistema de Gerenciamento de Inadimplentes. Cada ator possui responsabilidades específicas, que estão diretamente relacionadas ao uso e às funcionalidades disponibilizadas pela aplicação.

Responsável Financeiro

O Responsável Financeiro constitui-se como o principal usuário do sistema. Compete a este ator o cadastramento de clientes inadimplentes, o registro de pagamentos realizados, a atualização das informações financeiras e a emissão de relatórios gerenciais. Além disso, é incumbido de efetuar a autenticação no sistema, garantindo a segurança do acesso às informações. Sua participação é essencial para assegurar a confiabilidade dos dados e a efetividade do controle da inadimplência, uma vez que concentra a maior parte das operações críticas do sistema.

Proprietária do Escritório de Contabilidade

A Proprietária do Escritório de Contabilidade interage com o sistema de forma complementar, assumindo um papel de caráter gerencial. Suas atividades concentram-se na consulta a clientes inadimplentes, na visualização de relatórios financeiros e na análise de indicadores disponibilizados em dashboards. A partir destas informações, a proprietária obtém uma visão consolidada do cenário

financeiro do escritório, o que subsidia a definição de estratégias de cobrança e o planejamento administrativo

2.2 Modelos de Usuários

Nesta seção são apresentadas as personas que representam os principais usuários do Sistema de Gerenciamento de Inadimplentes. As personas foram elaboradas a partir da análise do contexto do escritório de contabilidade, dos processos de negócio e das necessidades identificadas no levantamento de requisitos.

A Tabela 1 descreve a persona do usuário Cláudia Pereira e Silva, proprietária e gestora administrativa do escritório. Nela é possível identificar que Cláudia sente falta de relatórios claros e consolidados, o que dificulta a identificação rápida dos clientes que mais possuem dívidas. Também é perceptível que ela busca acompanhar indicadores de inadimplência e ter uma visão consolidada da situação financeira. Por fim, após análise é possível ver que Cláudia deseja obter relatórios periódicos, dashboards com indicadores financeiros e uma visão geral da inadimplência que apoie suas decisões estratégicas.

Persona 1: Claudia Pereira	
Idade:	55 anos
Cargo:	Proprietária e gestora administrativa
Objetivos:	Acompanhar indicadores de inadimplência, ter uma visão consolidada da situação financeira e apoiar decisões estratégicas do escritório.
Frustrações:	Falta de relatórios claros e consolidados; dificuldade de identificar rapidamente os clientes que mais devem.
Necessidades:	Obter relatórios periódicos, dashboards com indicadores financeiros e visão geral da inadimplência.

Tabela 1. Persona Claudia Pereira.

A Tabela 2 descreve a persona do usuário José Carlos, responsável pelo setor financeiro. Nela é possível identificar que José, apesar de ser experiente na área financeira, ainda depende de anotações manuais em cadernos, o que aumenta o risco de falhas e dificulta a atualização em tempo real dos registros. Também é perceptível que ele busca reduzir erros manuais e gerar relatórios confiáveis para apoiar a gestão do escritório. Por fim, após análise é possível ver que José deseja um sistema simples, seguro e de fácil usabilidade, que permita cadastrar inadimplentes, registrar pagamentos e emitir relatórios precisos.

Persona 2: José Carlos	
Idade:	55 anos
Cargo:	Responsável pelo setor financeiro
Objetivos:	Manter controle atualizado sobre clientes inadimplentes; reduzir erros manuais; gerar relatórios para apoiar a gestão.
Frustrações:	Atualmente depende de anotações em cadernos, o que aumenta o risco de falhas, dificulta a atualização em tempo real e compromete a precisão dos registros.
Necessidades:	Sistema simples, seguro e de fácil usabilidade, que permita cadastrar inadimplentes, registrar pagamentos e emitir relatórios confiáveis.

Tabela 2. Persona José Carlos.

2.3 Modelo de Casos de Uso e Histórias de Usuários

Esta subseção tem como objetivo descrever os casos de uso e as histórias de usuário previstos para o projeto. Para tanto, apresenta-se um diagrama de casos de uso em que todos são listados. De forma complementar, também são apresentadas as histórias de usuário relacionadas às funcionalidades previstas para o sistema a ser desenvolvido.

2.3.1 Diagrama de caso de uso

O diagrama de casos de uso (Figura 1) representa as funcionalidades do sistema financeiro do escritório, com dois atores principais: o Responsável Financeiro e a Proprietária do Escritório. O Responsável Financeiro realiza operações como registro de pagamentos, consulta e cadastro de clientes inadimplentes, geração de relatórios, exportação de dados, autenticação e visualização de indicadores além de agendar a periodicidade do envio do email ao cliente. A ação de registrar pagamento inclui automaticamente a atualização do saldo devedor. Já a Proprietária do Escritório possui acesso à visualização do dashboard e à geração de relatórios, com foco em análise e tomada de decisão.

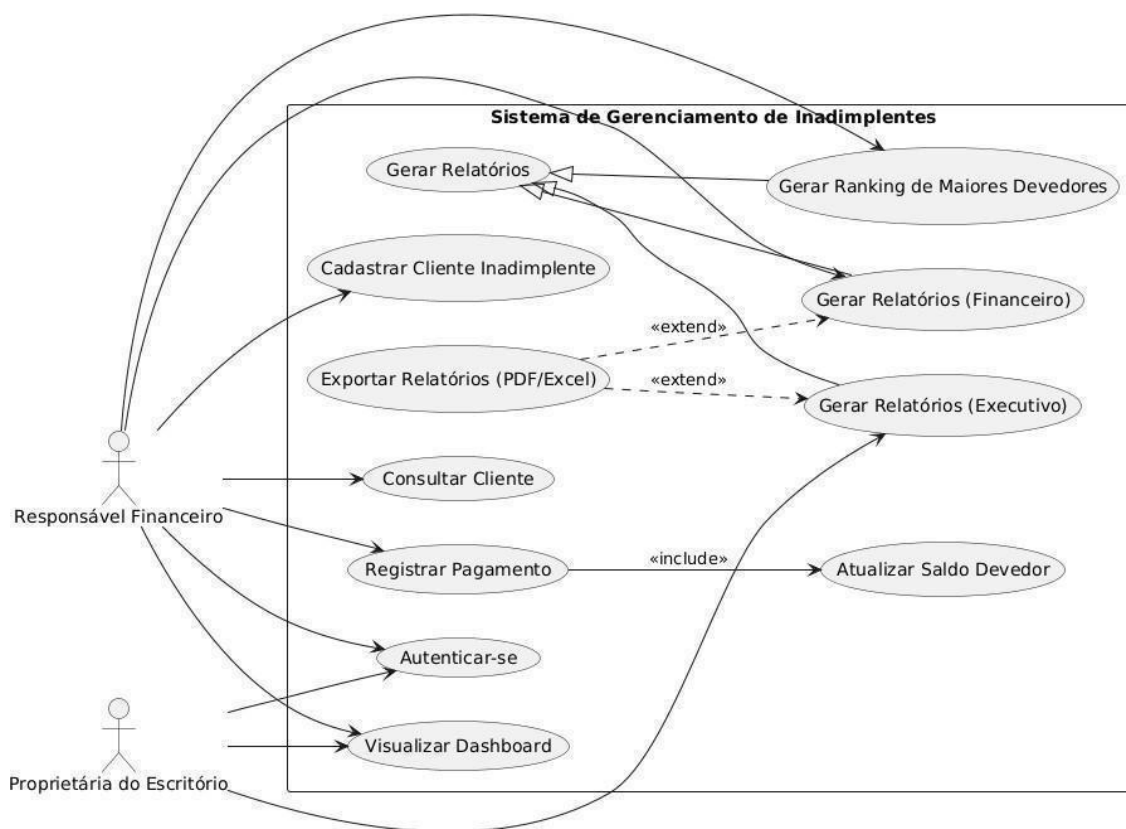


Figura 1. Diagrama de Caso de Uso

2.3.2 Histórias do Usuário

Nesta seção são listadas as histórias de usuários elucidadas para o sistema financeiro proposto. Para fins de organização, utiliza-se o identificador no formato US#ID, em que US refere-se a *User Story*.

As histórias de usuários identificadas para o sistema são:

US1. Como responsável financeiro, gostaria de registrar pagamentos de clientes para manter o controle atualizado das pendências.

US2. Como responsável financeiro, eu gostaria de atualizar automaticamente o saldo devedor ao registrar um pagamento, para garantir consistência nas informações.

US3. Como responsável financeiro, eu gostaria de cadastrar clientes inadimplentes para controlar melhor os casos de atraso.

US4. Como responsável financeiro, eu gostaria de gerar relatórios financeiros para análise interna.

US5. Como responsável financeiro, eu gostaria de exportar relatórios em PDF ou Excel para facilitar o compartilhamento e arquivamento.

US6. Como responsável financeiro, eu gostaria de visualizar o *ranking* de maiores devedores para identificar os principais inadimplentes.

US7. Como responsável financeiro, eu gostaria de consultar dados dos clientes para facilitar o atendimento e a tomada de decisões.

US8. Como responsável financeiro, eu gostaria de gerenciar autenticação para garantir o acesso seguro ao sistema.

US9. Como proprietária do escritório, eu gostaria de visualizar um *dashboard* com indicadores financeiros para acompanhar a saúde do negócio.

US10. Como proprietária do escritório, eu gostaria de gerar relatórios sempre que necessário para entender o desempenho financeiro da empresa.

US11. Como responsável financeiro, eu gostaria de enviar automaticamente e-mails para clientes inadimplentes com o valor devido, para agilizar o processo de cobrança.

US12. Como responsável financeiro, eu gostaria de agendar envios periódicos de lembretes, para clientes com débitos em atraso.

2.4 Diagrama de Sequência do Sistema e Contrato de Operações

Esta seção descreve os principais fluxos de interação do SGI – Sistema de Gerenciamento de Inadimplentes por meio de diagramas de sequência, detalhando a ordem das mensagens entre App (usuário/Responsável Financeiro), Backend (API) e serviços de domínio. Cada fluxo possui um Contrato de Operação com pré-condições, pós-condições e exceções. As legendas e referências seguem o padrão do documento.

O fluxo de autenticação inicia quando o usuário informa suas credenciais no app; o *backend* delega a verificação ao AuthService, que consulta o *UsuarioRepository*; com credenciais válidas, o serviço emite o tokenSessao e identifica o perfil; por fim, o *backend* retorna 200 OK ao app com o *token* e o perfil do usuário.

Contrato	Autenticar Usuário
Operação	autenticar(telefone: String, senhaOuOTP: String) -> token: String, perfil: Perfil
Referências cruzadas	US8 – Gerenciar autenticação (acesso seguro ao sistema)
Pré-condições	App online; credenciais informadas; usuário existente/ativo.
Pós-condições	Sessão ativa com tokenSessao válido; perfil associado; auditoria “login_sucesso”.

A Figura 2 apresenta o diagrama de sequência referente ao processo de autenticação do usuário no SGI. O fluxo inicia quando o usuário informa suas credenciais de acesso, como telefone e senha. Em seguida, o sistema realiza a busca do usuário cadastrado, valida as credenciais informadas e, caso estejam corretas, emite um token de sessão associado ao perfil do usuário. Ao final, o sistema retorna uma resposta de sucesso contendo o token e as informações necessárias para permitir o acesso às funcionalidades da aplicação.

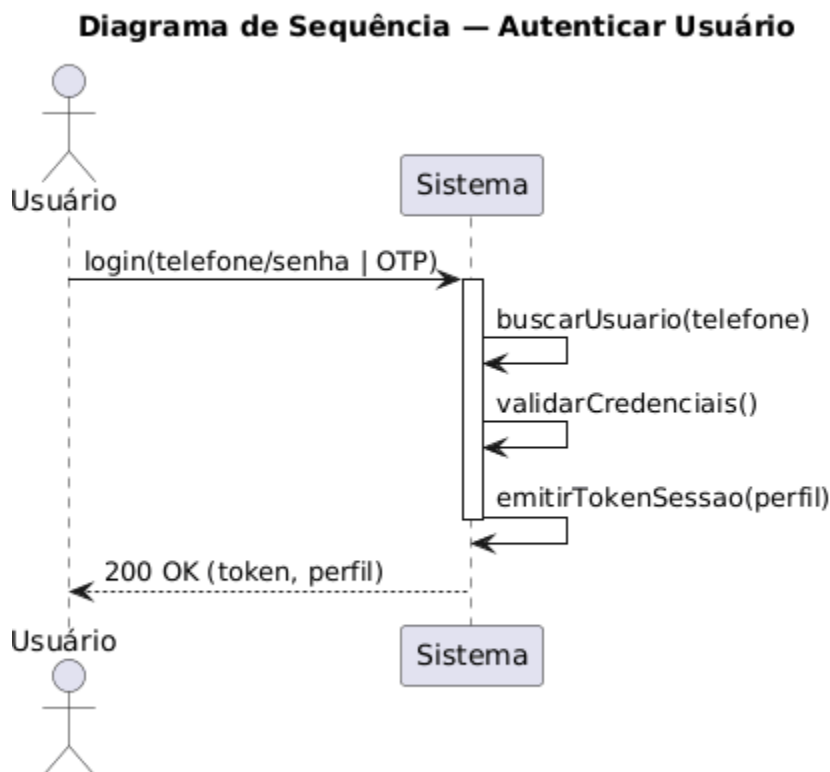


Figura 2 - Autenticar Usuario

Conforme será mostrado na Figura 3. O Responsável Financeiro seleciona o cliente e envia os dados do pagamento; no backend, o PagamentoService é acionado para recuperar o cliente, persistir o pagamento e, no mesmo fluxo transacional, recalculando o saldo devedor no repositório; ao final, a API retorna ao app o recibo e o novo saldo confirmado.

Contrato	Registrar Pagamento
Operação	registrarPagamento(clienteId: UUID, valor: Money, data: Date, comprovante: Blob) -> recibo: PDF, novoSaldo: Money
Referências cruzadas	US1 e US2
Pré-condições	Usuário autenticado; clienteId existente/ativo; valor > 0; data ≤ hoje.
Pós-condições	Pagamento persistido; saldo recalculado; recibo emitido; auditoria "pagamento_registrado".

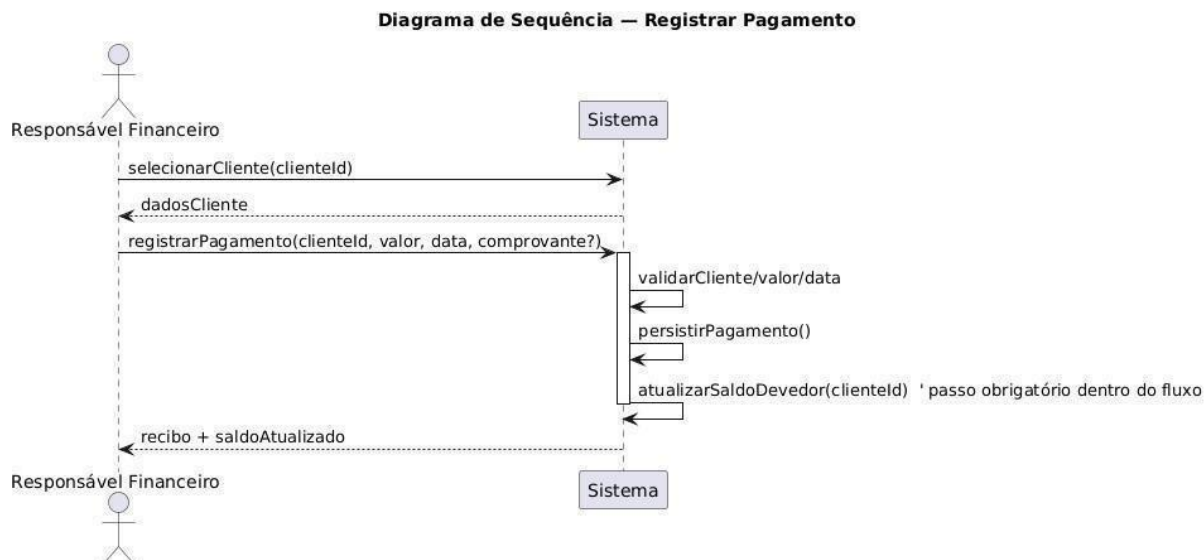


Figura 3 - Registrar Pagamento

Conforme será mostrado na Figura 4, para um cliente previamente validado o responsável financeiro registra uma nova inadimplência informando valor, vencimento e descrição; o InadimplenciaService aplica as regras de negócio, valida os dados e persiste o lançamento com status inicial “Em Aberto”, gerando o respectivo protocolo; em seguida, o backend confirma ao aplicativo a criação do registro (incluindo o identificador/protocolo) e registra a trilha de auditoria do evento, com data-hora, usuário e origem da operação.

Contrato	Registrar Inadimplência
Operação	registrarInadimplencia(clientId: UUID, valor: Money, vencimento: Date, descricao: String) -> protocolo: String
Referências cruzadas	US3
Pré-condições	clientId existente/ativo; valor > 0; vencimento ≥ hoje; usuário autenticado.
Pós-condições	Inadimplência criada (status = EmAberto); auditoria “inadimplencia_registrada”.

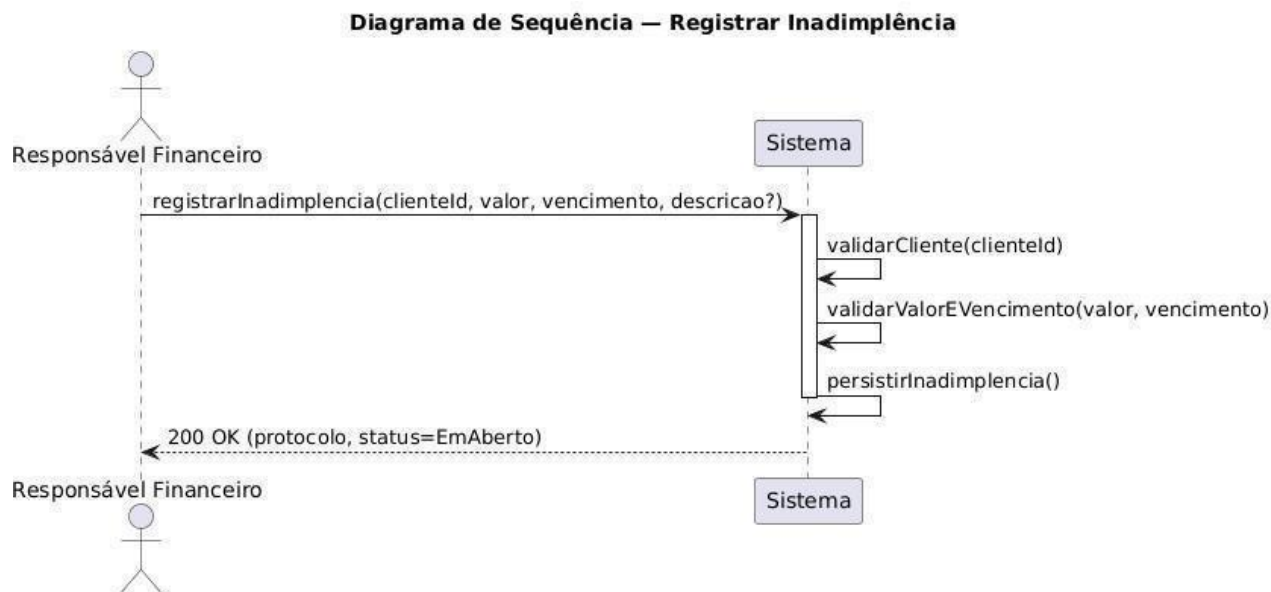


Figura 4 — Registrar Inadimplência.

Conforme será mostrado na Figura 5, o usuário solicita a geração de um relatório (por exemplo, ranking de maiores devedores ou resumo por período), parametrizando os filtros desejados; o `RelatorioService` consulta o repositório, consolida os dados e retorna o dataset/visão para exibição no aplicativo; opcionalmente, o usuário pode acionar a exportação em PDF ou Excel, gerando o arquivo correspondente para download e arquivamento.

Contrato	Gerar Relatório
Operação	<code>gerarRelatorio(tipo: Enum, periodo: {ini, fim}, filtros?: Map) -> view/dataset</code>
Referências cruzadas	US4,US5,US6
Pré-condições	Usuario autenticado; filtros válidos.
Pós-condições	Resultado exibido; se exportado, arquivo gerado (PDF/Excel) e link disponibilizado

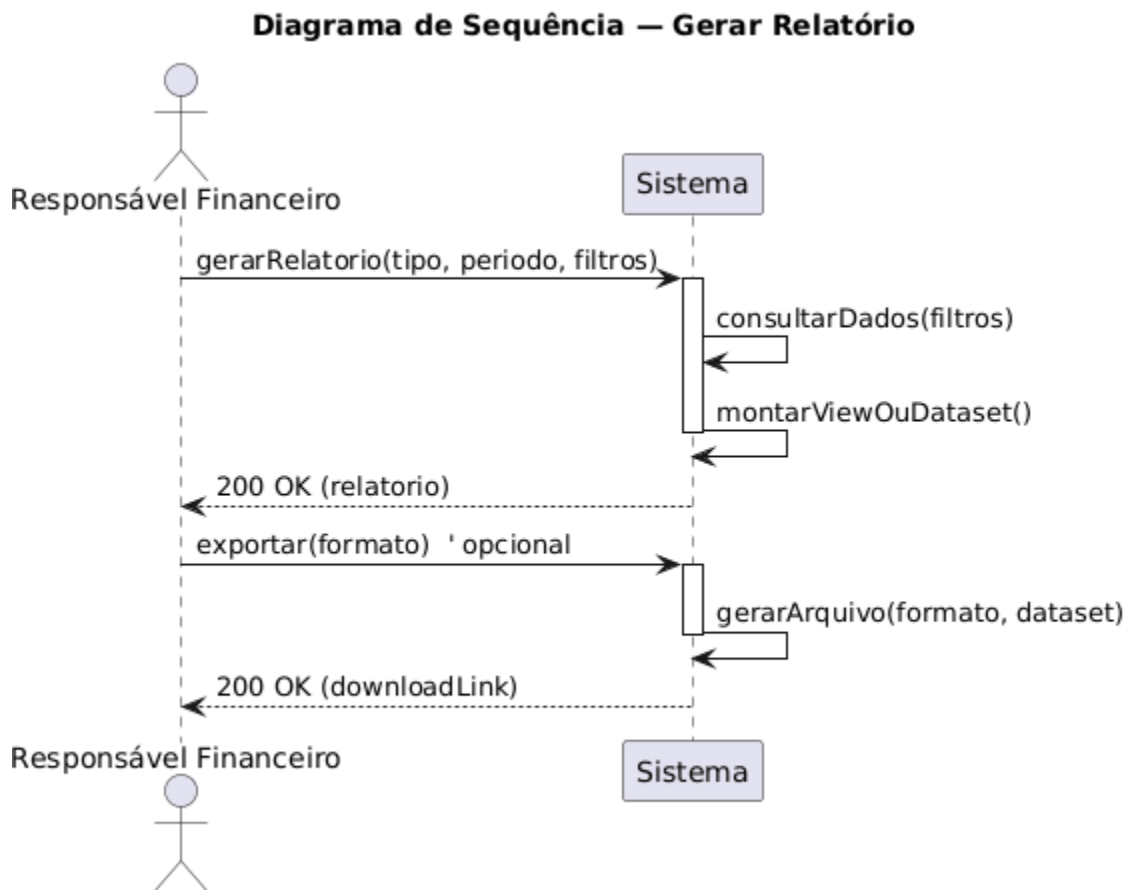


Figura 5 — Gerar Relatório.

Conforme será mostrado na Figura 6, o fluxo de recálculo de saldo consolida os débitos em aberto e os pagamentos efetivados de um cliente, podendo ser acionado manualmente pela interface do usuário ou automaticamente como processo interno subsequente à confirmação de pagamento; ao término da execução, o novo saldo é persistido no repositório de dados e retornado ao aplicativo, assegurando a consistência entre os lançamentos registrados e a visão financeira apresentada ao usuário.

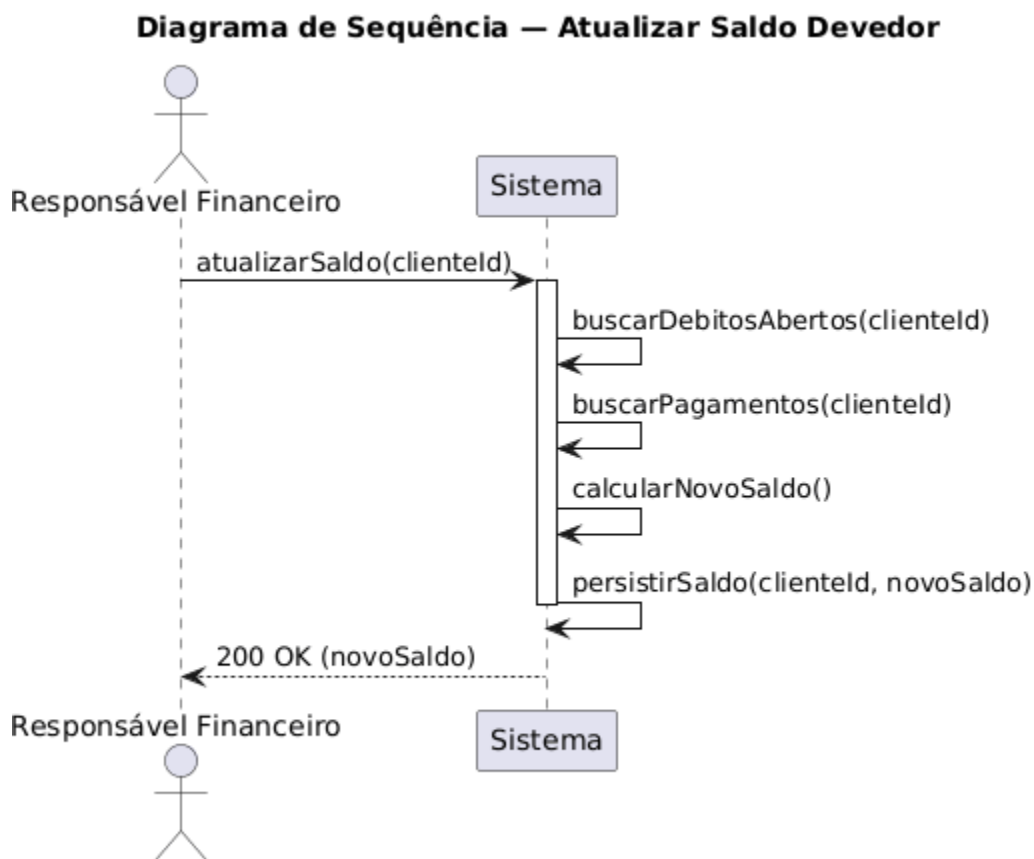


Figura 6 — Atualizar saldo devedor

Contrato	Atualizar Saldo Devedor
Operação	atualizarSaldoDevedor(clientId)
Referências cruzadas	US2, atualizar saldo devedor.
Pré-condições	Usuário autenticado (na execução manual); <i>clienteId</i> válido.
Pós-condições	novoSaldo calculado e persistido; auditoria “saldo_recalculado”.

3. Modelos de Projeto

Esta seção apresenta os modelos de projeto do sistema proposto, organizados nas seguintes subseções: Visão Geral dos Modelos (Seção 3.1), Diagrama de Classes (Seção 3.2), Diagramas de Sequência (Seção 3.3), Diagrama de Comunicação (Seção 3.4), Arquitetura (Seção 3.5), Diagramas de Estados (Seção 3.6) e Diagramas de Componentes e Implantação (Seção 3.7). Em conjunto, esses artefatos oferecem uma visão integrada das estruturas, dos comportamentos e da distribuição do sistema.

3.1 Diagrama de Classes

A Figura 7 apresenta o diagrama de classes do núcleo de domínio do SGI — Sistema de Gerenciamento de Inadimplentes. O modelo contempla as principais entidades responsáveis pelo controle financeiro do escritório: Cliente, Inadimplência, Pagamento e Usuário. A classe Cliente representa as pessoas físicas ou jurídicas cadastradas no sistema, contendo informações como nome, CPF/CNPJ, e-mail, telefone, status e data de criação. Cada cliente pode possuir uma ou mais inadimplências associadas. A classe Inadimplência representa um débito registrado para determinado cliente, contendo informações como valor original, saldo devedor, data de vencimento, descrição e status. O status da inadimplência pode assumir valores como EM_ABERTO, PARCIAL, QUITADA ou VENCIDA, permitindo acompanhar o ciclo de vida financeiro do débito.

A classe Pagamento registra os valores pagos pelo cliente, contendo data de pagamento, valor, forma de pagamento e vínculo com a inadimplência correspondente. Uma inadimplência pode receber um ou mais pagamentos, permitindo quitações parciais ou totais. A classe Usuário representa os usuários internos do sistema, como o responsável financeiro e a proprietária do escritório, contendo dados de autenticação, perfil de acesso e status. Esse controle é necessário para garantir segurança no acesso às funcionalidades da aplicação.

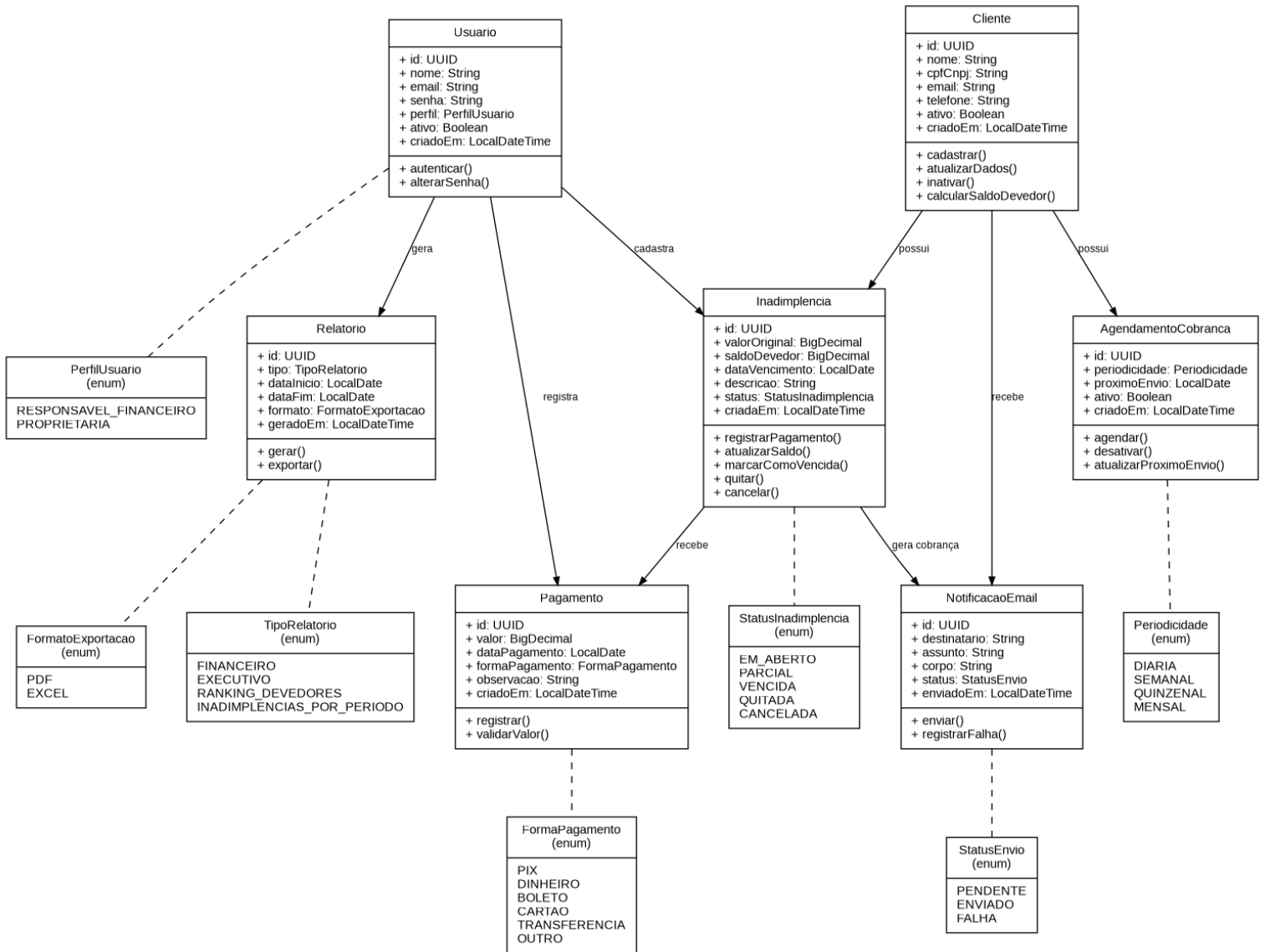


Figura 7 — Diagrama de Classes

3.2 Diagramas de Sequência

A Figura 8 apresenta o diagrama de sequência de Autenticar o Usuário. O fluxo inicia quando a usuária informa suas credenciais no frontend, que envia uma requisição POST para o endpoint de autenticação da API. Em seguida, o AuthController delega a validação ao AuthService, que consulta o UsuarioRepository para localizar o usuário pelo e-mail informado. Caso o usuário não exista ou as credenciais sejam inválidas, o sistema retorna erro 401 Unauthorized. Caso os dados estejam corretos, o serviço valida a senha, gera um token JWT com as informações de perfil e retorna uma resposta 200 OK ao frontend, permitindo o acesso à tela principal do sistema.

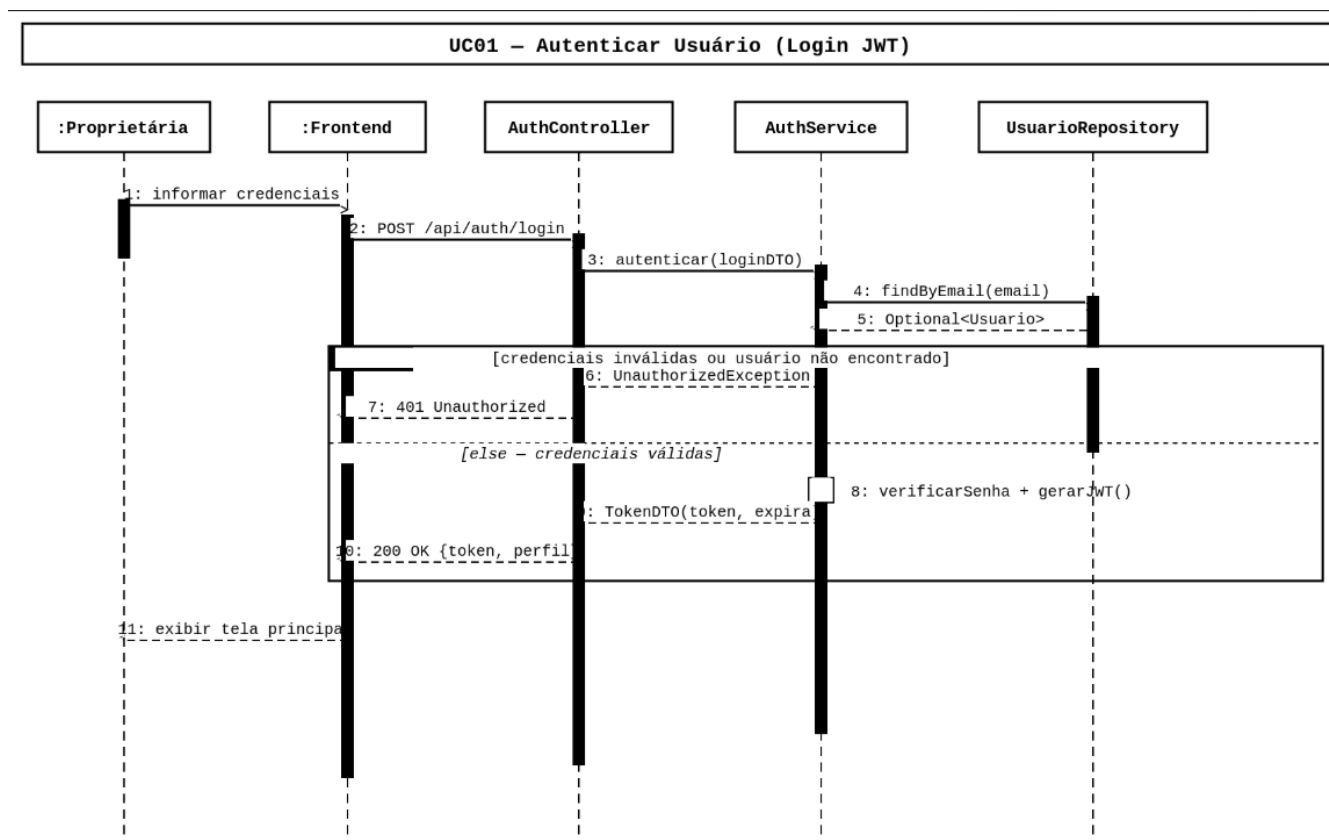


Figura 8 — Diagrama de Sequência.

A Figura 9 apresenta o diagrama de sequência Gerenciar Clientes. O fluxo contempla as operações de cadastro e consulta de clientes no SGI. No cadastro, a usuária preenche os dados do cliente no frontend, que envia uma requisição POST para a API. O ClienteController aciona o ClienteService, responsável por validar as informações e verificar se o CPF já está cadastrado. Caso exista duplicidade, o sistema retorna erro 422 Unprocessable Entity. Se os dados forem válidos, o ClienteRepository persiste o novo cliente e retorna uma resposta 201 Created. Na consulta, o frontend envia uma requisição GET com os filtros informados, e o sistema retorna uma lista de clientes cadastrados, mesmo que vazia.

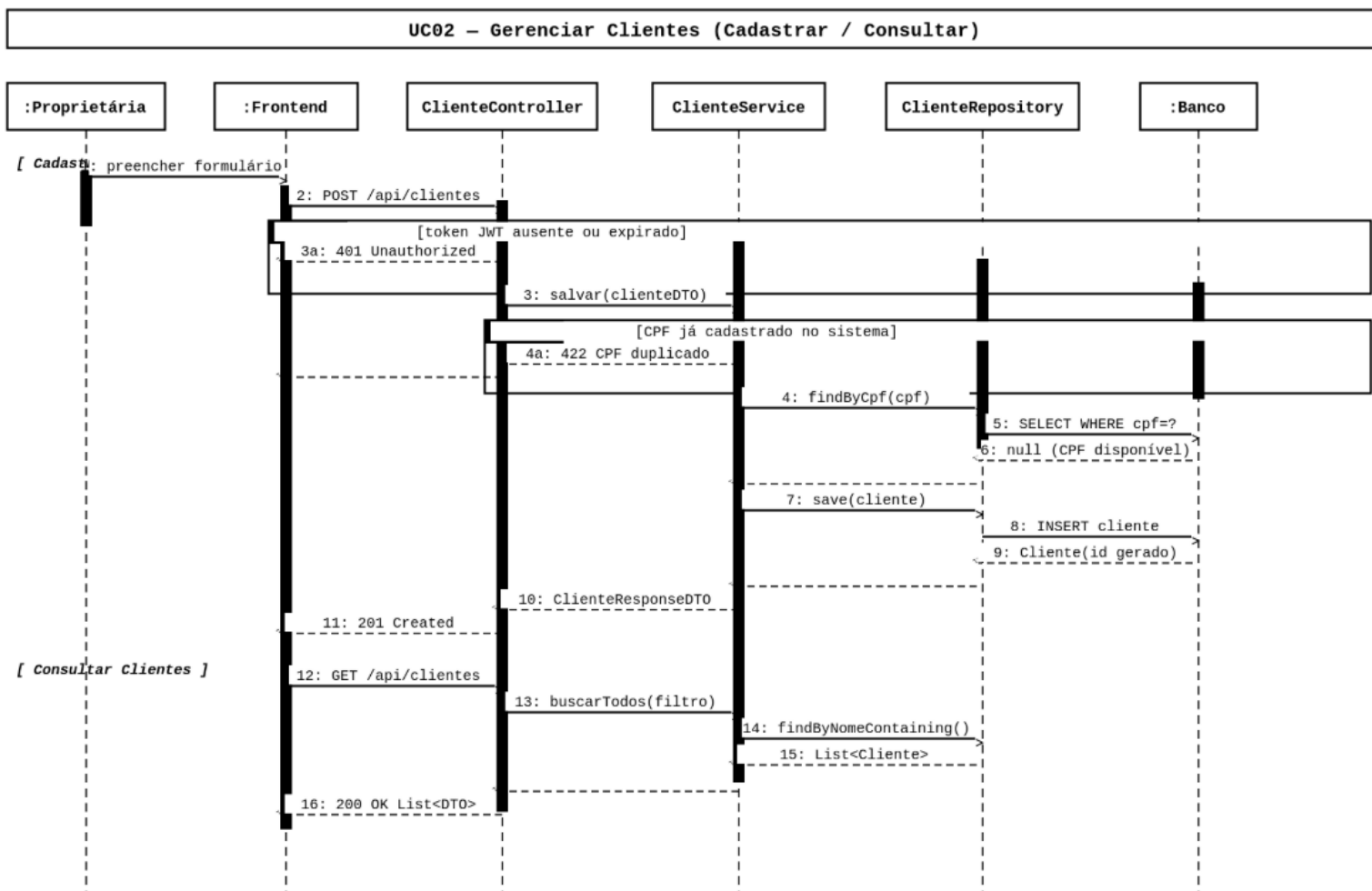


Figura 9— Diagrama de Sequência.

A Figura 10 apresenta o diagrama de sequência do caso de uso UC03 — Gerenciar Dívidas. O fluxo descreve o registro e a consulta de dívidas vinculadas a um cliente. Para registrar uma dívida, a usuária informa os dados necessários, como cliente, valor, vencimento e descrição. O frontend envia a requisição para o DividaController, que delega a operação ao DividaService. O serviço verifica se o cliente informado existe por meio do ClienteRepository. Caso o cliente não seja encontrado, o sistema retorna erro 404 Not Found. Caso contrário, a dívida é persistida com status inicial ABERTA e o sistema retorna 201 Created. O diagrama também contempla a consulta das dívidas de um cliente, retornando uma lista com os registros encontrados ou uma lista vazia quando não houver dívidas cadastradas.

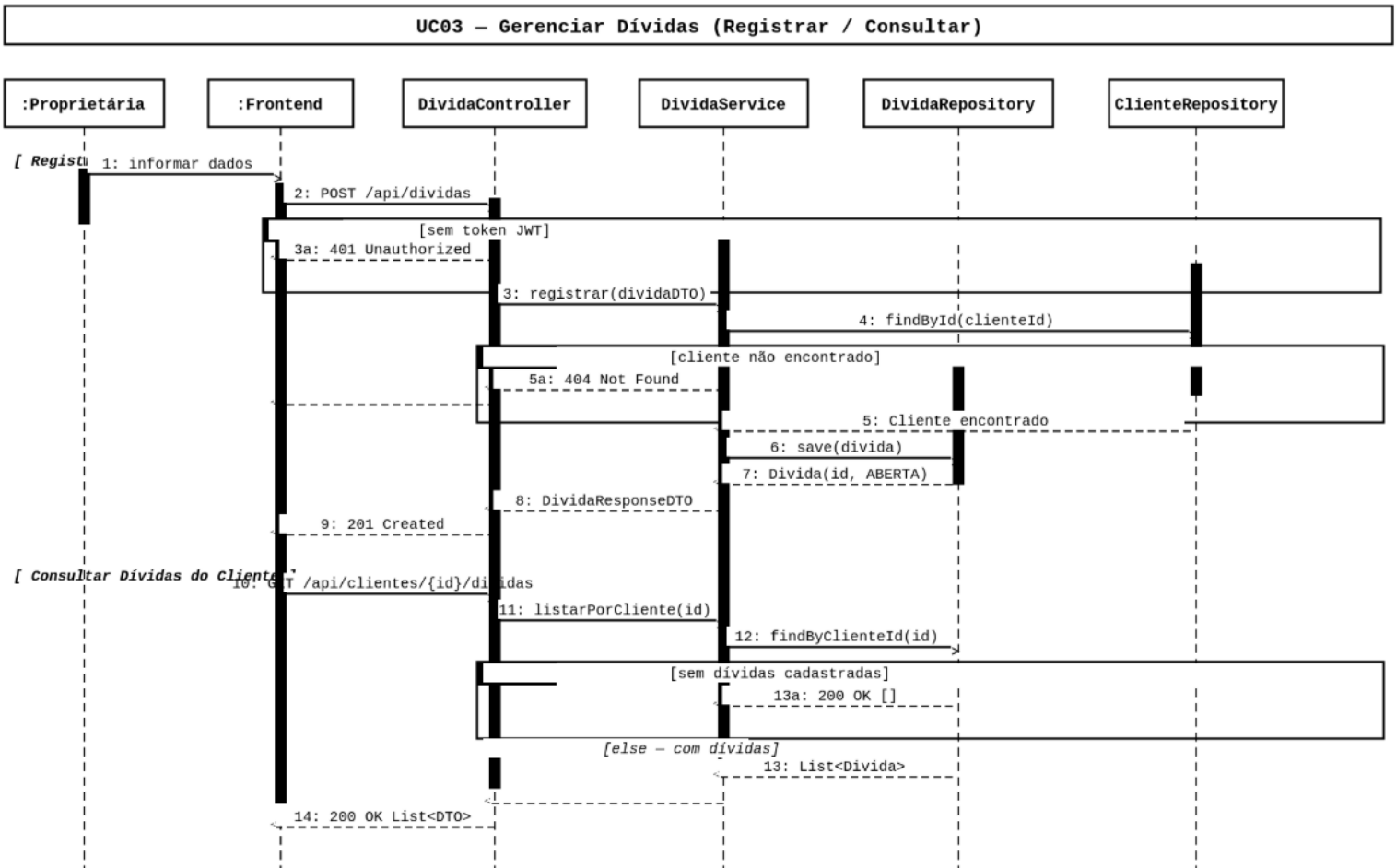


Figura 10— Diagrama de Sequência.

A Figura 11 apresenta o diagrama de sequência do caso de uso UC04 — Registrar Pagamento de Dívida. Esse fluxo representa o processo de baixa parcial ou total de uma dívida no SGI. A usuária seleciona uma dívida e informa o valor do pagamento no frontend, que envia a requisição para a API. O PagamentoController aciona o PagamentoService, responsável por buscar a dívida, validar o valor informado e recalculer o saldo devedor. Caso a dívida não exista, é retornado erro 404 Not Found. Caso o valor seja inválido, menor ou igual a zero, ou superior ao saldo devedor, o sistema retorna erro 422 Unprocessable Entity. Quando os dados são válidos, o pagamento é salvo, a dívida é atualizada com status PARCIAL ou QUITADA e o sistema retorna um ReciboDTO com resposta 201 Created.

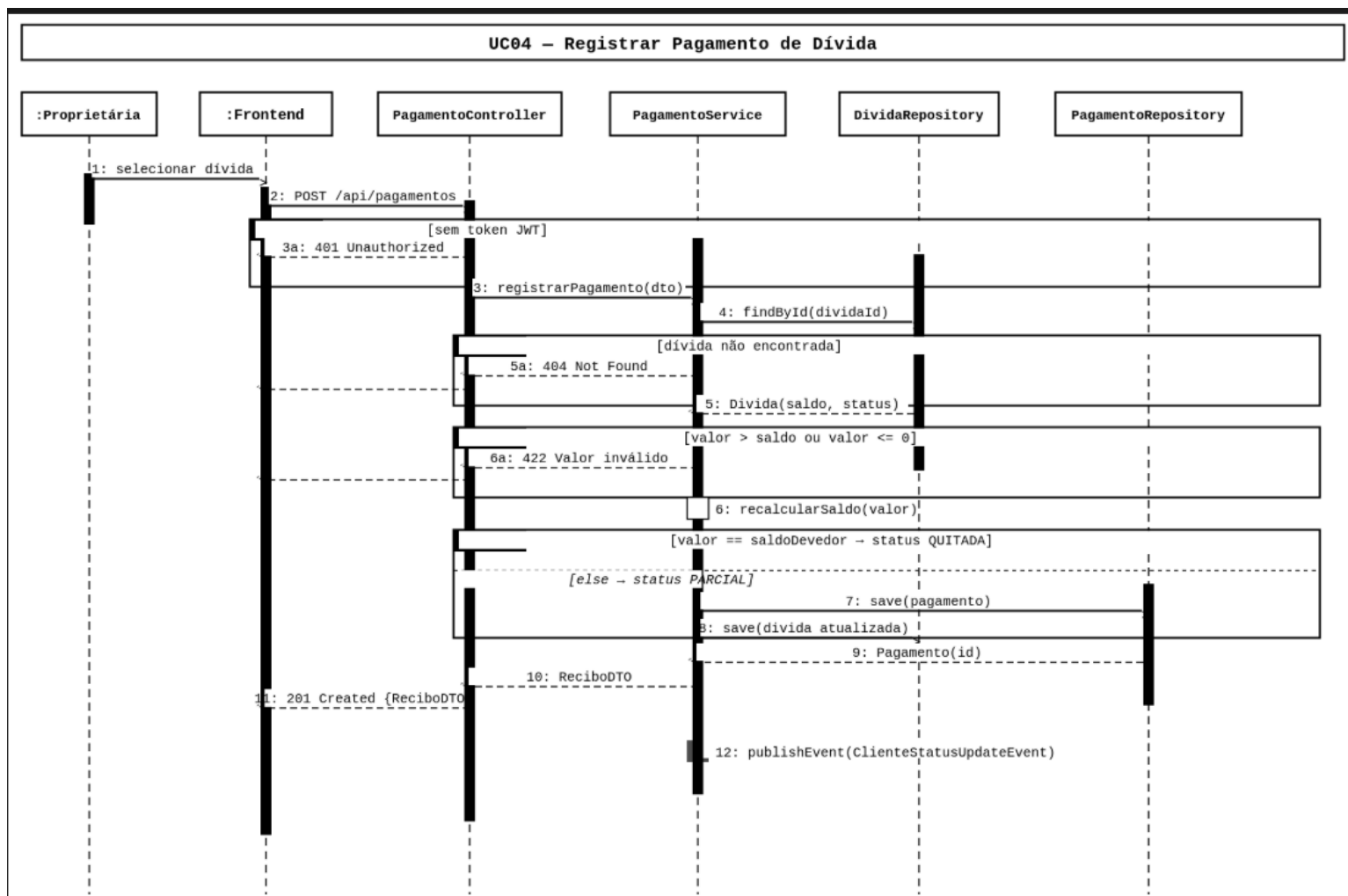


Figura 11— Diagrama de Sequência.

A Figura 12 apresenta o diagrama de sequência de Gerar e Enviar Cobrança. O fluxo descreve o envio de cobrança por e-mail para clientes com dívidas em aberto. A usuária seleciona o cliente no frontend e solicita a geração da cobrança. O CobrancaController encaminha a requisição ao CobrancaService, que verifica se existem dívidas abertas associadas ao cliente. Caso não existam débitos em aberto, o sistema retorna erro 422 Unprocessable Entity. Se houver dívidas válidas, o serviço compõe a mensagem de cobrança, registra o envio no banco de dados e utiliza o servidor SMTP para enviar o e-mail. Em caso de falha no envio, é retornado erro 503 Service Unavailable. Em caso de sucesso, o sistema retorna 201 Created com as informações da cobrança gerada.

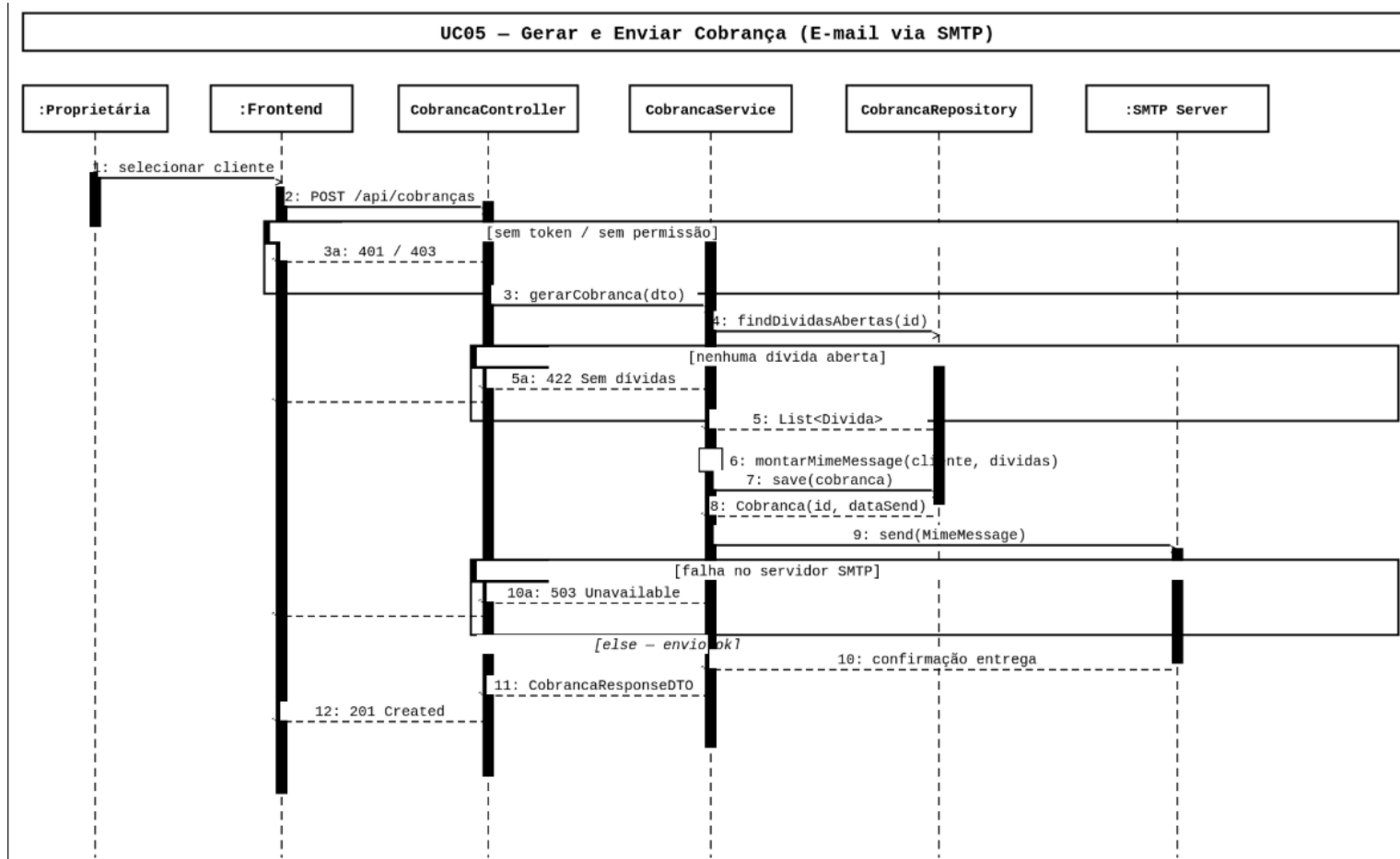


Figura 12— Diagrama de Sequência.

A Figura 13 apresenta o diagrama de sequência do caso de uso UC06 — Gerar Relatório de Inadimplência. Esse fluxo representa a geração de relatórios gerenciais para acompanhamento da situação financeira dos clientes inadimplentes. A Proprietária informa filtros de período e status no frontend, que envia uma requisição GET para a API. O RelatorioController aciona o RelatorioService, responsável por consultar as dívidas e os pagamentos no período informado. Com base nesses dados, o serviço calcula indicadores como total em aberto, total arrecadado e taxa de inadimplência. Caso não existam registros no período, o sistema retorna um RelatorioDTO vazio com resposta 200 OK. Caso existam dados, retorna o relatório preenchido para exibição no frontend.

UC06 – Gerar Relatório de Inadimplência

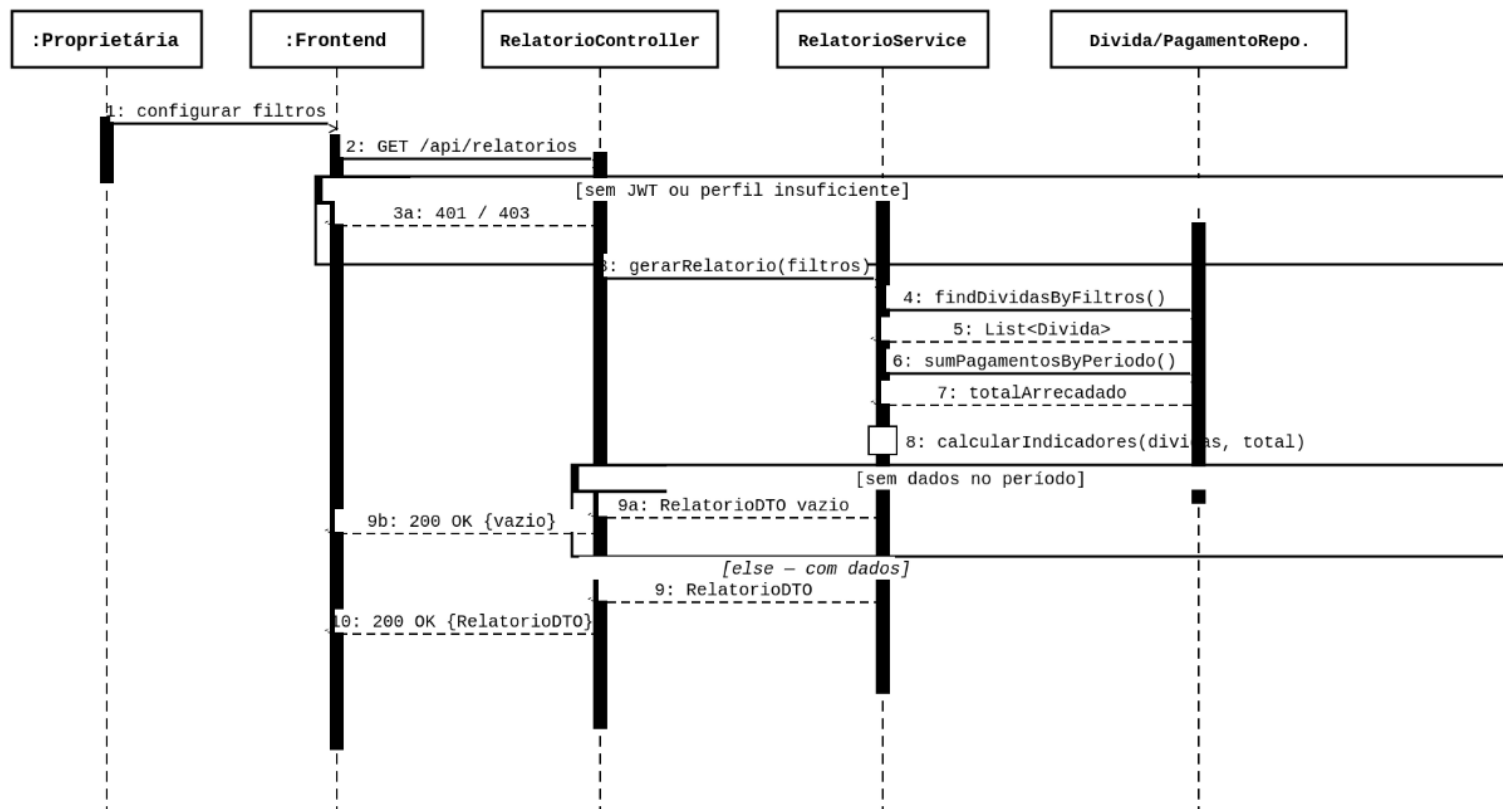


Figura 13— Diagrama de Sequência.

A Figura 14 apresenta o diagrama de sequência do caso de uso UC07 — Gerenciar Usuários. Esse fluxo é restrito à Proprietária e contempla as operações de criação, alteração de perfil e revogação de acesso de usuários do sistema. Para criar um novo usuário, a Proprietária informa nome, e-mail, senha e perfil no frontend. A API valida o token JWT e verifica se o perfil possui permissão para executar a operação. Caso o usuário não tenha permissão, o sistema retorna erro 403 Forbidden. Em seguida, o UsuarioService verifica se o e-mail já está cadastrado; em caso positivo, retorna erro 422 Unprocessable Entity. Caso os dados sejam válidos, a senha é criptografada, o usuário é persistido e o sistema retorna 201 Created. O fluxo também contempla a alteração de perfil ou desativação do usuário, preservando o histórico das operações realizadas no sistema.

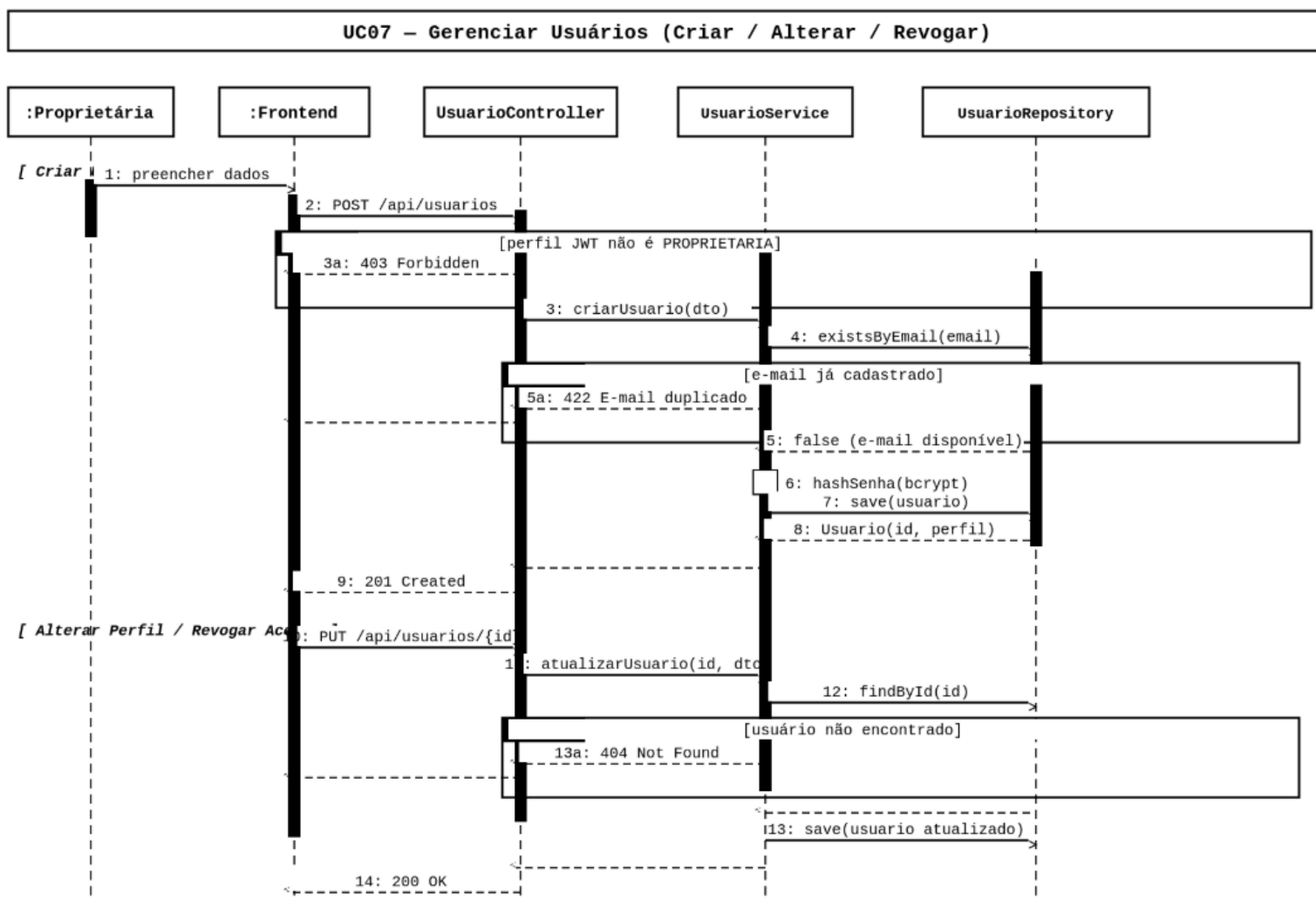


Figura 14— Diagrama de Sequência.

3.3 Diagramas de Comunicação

Nesta seção, são apresentados os diagramas de comunicação que modelam as trocas de mensagens entre atores, camadas e serviços do SGI – Sistema de Gerenciamento de Inadimplentes. O objetivo é documentar quem se comunica com quem e em que ordem, cobrindo os principais fluxos: autenticação, registro de pagamento com atualização de saldo, consulta de clientes/inadimplências e geração/expedição de relatórios. Esses diagramas complementam os diagramas de sequência e servem como referência dos contratos de mensagem da aplicação.

A Figura 10 apresenta as mensagens envolvidas no registro de pagamento de um cliente inadimplente. O Responsável Financeiro inicia o processo pela UI-Web, que valida a sessão no AuthService e encaminha a solicitação ao PagamentoService. Esse consulta o ClienteService para obter o cliente, persiste o pagamento no Repositório e, obrigatoriamente, dispara a atualização do saldo devedor (comportamento interno reutilizado). Ao final, o serviço retorna a confirmação e recibo à interface, que exibe o resultado ao usuário.

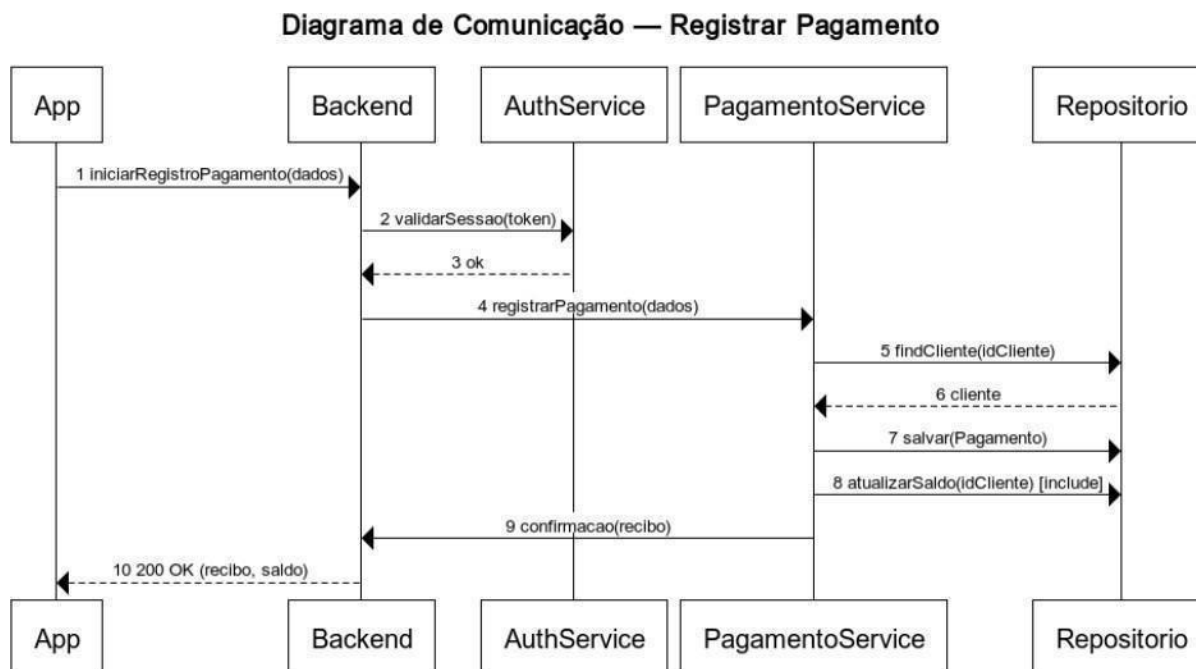


Figura 15 — Diagrama de Comunicação

Figura 11 – Diagrama de Comunicação: Gerar Relatórios (Financeiro) e Exportar (PDF/Excel)

A Figura 11 descreve o fluxo de geração de relatórios financeiros. O Responsável Financeiro solicita o relatório à UI-Web, que chama o RelatorioService. O serviço consulta dados no Repositório e devolve o conjunto resultante para exibição. Opcionalmente, quando o usuário seleciona Exportar, a interface aciona o *ExportService* para produzir o arquivo em PDF ou Excel, disponibilizando o download ao término.

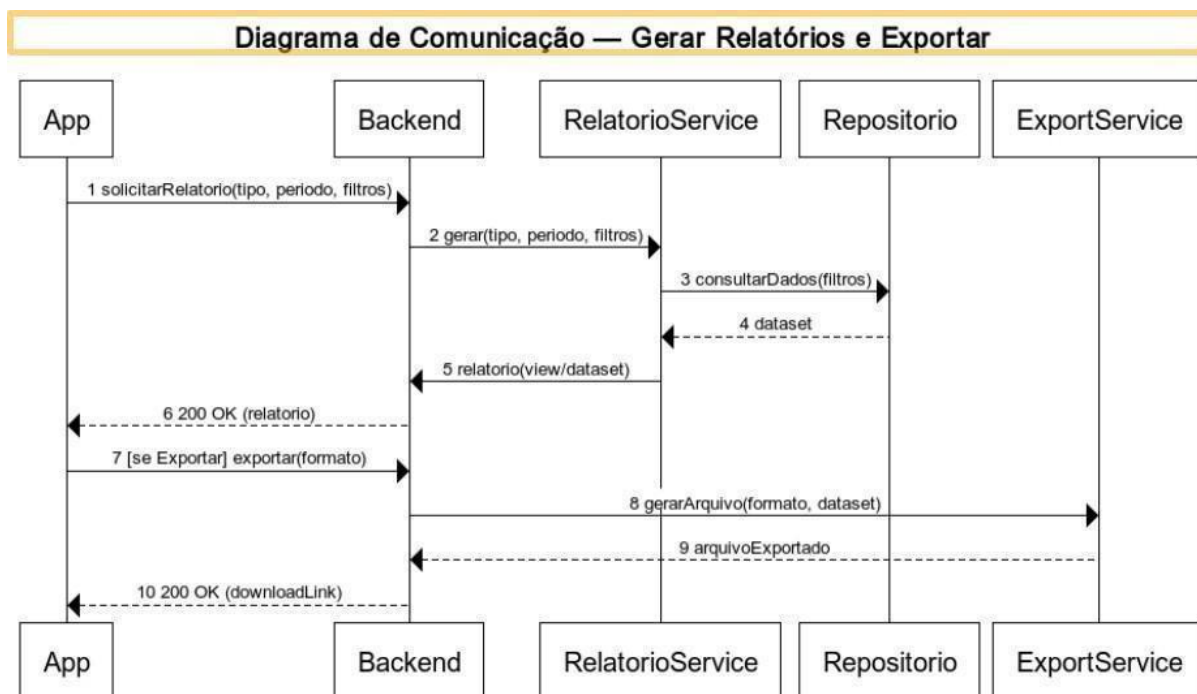


Figura 16 — Diagrama de Comunicação

Figura 12 – Diagrama de Comunicação: Consultar Cliente/Inadimplência (com cache e iteração)

A Figura 12 mostra as mensagens trocadas na consulta de clientes e suas inadimplências. O Responsável Financeiro solicita a busca à UI-Web, que encaminha ao ClienteService. Quando habilitado, o serviço consulta primeiro o Cache; em caso de cache miss, realiza query no Repositório. Em seguida, para cada cliente retornado, o serviço itera a obtenção dos detalhes de inadimplência, consolidando o resultado para exibição.

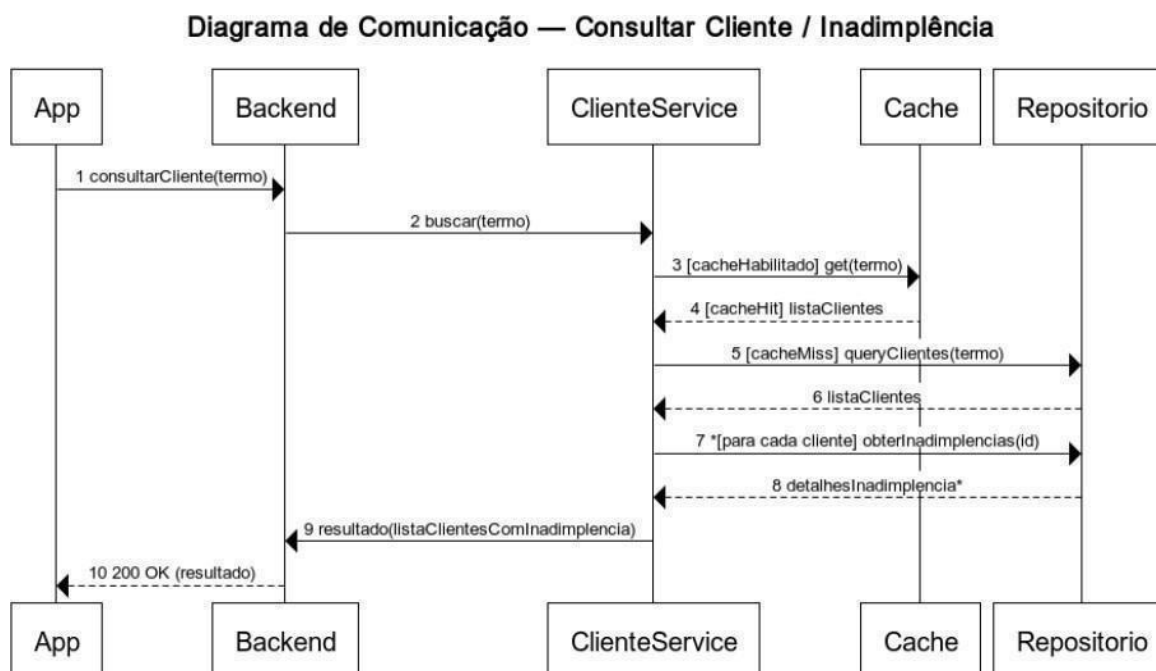


Figura 17 — Diagrama de Comunicação

3.4 Arquitetura

A arquitetura do sistema está organizada em três camadas principais — UI, API e Data — com separação explícita de responsabilidades. A solução será disponibilizada em ambiente web, não havendo necessidade de instalação local na máquina do cliente. Dessa forma, os usuários poderão acessar o sistema por meio de um navegador, utilizando uma URL pública da aplicação.

Na camada de UI, o sistema contempla um Frontend Web desenvolvido com React + Vite, hospedado na plataforma Vercel. Essa camada é responsável pela interação com os usuários, exibindo as telas do sistema e capturando ações como autenticação, cadastro de clientes, registro de inadimplências, registro de pagamentos, consultas, visualização de dashboards e geração de relatórios. A comunicação com a API ocorre por meio de requisições HTTP/HTTPS no formato JSON, seguindo o padrão REST.

A escolha do React + Vite para o frontend foi motivada pela produtividade no desenvolvimento de interfaces modernas, pela organização em componentes reutilizáveis e pela facilidade de integração com APIs REST. Já a hospedagem na Vercel foi escolhida por oferecer deploy simplificado para aplicações frontend, integração com repositórios Git, distribuição eficiente de

arquivos estáticos e atualização rápida a cada nova versão da aplicação.

A camada de API é implementada em Spring Boot com Java 21 e será hospedada na plataforma Render. Essa camada concentra a orquestração das regras de negócio e a exposição dos endpoints REST utilizados pelo frontend. Optou-se pelo Spring Boot devido à sua robustez, maturidade e facilidade de integração com recursos como JPA/Hibernate, validação de dados, autenticação e documentação de endpoints. Nessa camada, os Controllers atuam como pontos de entrada da aplicação, recebendo as requisições do frontend, encaminhando-as para os Services e retornando respostas padronizadas.

Os Services centralizam as regras de negócio do sistema, como validação de pagamentos, atualização de saldo devedor, alteração do status da inadimplência e consolidação de dados para relatórios. O acesso aos dados é realizado por meio dos Repositories, utilizando Spring Data JPA, que abstrai consultas e operações de persistência. Essa organização favorece a coesão dos módulos, reduz o acoplamento entre as camadas e facilita a evolução do sistema.

Para o processo de implantação do backend, será utilizado Docker, permitindo empacotar a aplicação Spring Boot com suas dependências em um ambiente padronizado. O uso do Docker contribui para reduzir problemas de configuração entre ambientes de desenvolvimento e produção, além de facilitar o deploy no Render. Com isso, a API pode ser executada de forma mais previsível, independentemente das diferenças entre máquinas ou ambientes.

Na camada de Data, o sistema utiliza um banco de dados acessado pela API hospedada no Render. O banco será responsável por armazenar as informações de clientes, inadimplências, pagamentos, usuários e demais registros necessários ao funcionamento do SGI. Em ambiente de desenvolvimento, poderá ser utilizado SQLite pela sua simplicidade e facilidade de configuração. Para o ambiente hospedado, recomenda-se o uso de um banco persistente, como PostgreSQL, ou uma configuração com disco persistente no Render, garantindo que os dados não sejam perdidos em reinicializações do serviço.

Esse arranjo em UI → API → Data promove alta coesão dentro de cada módulo e baixo acoplamento entre as camadas. A separação entre frontend, backend e persistência melhora a testabilidade, a manutenibilidade e a escalabilidade do sistema. Além disso, a hospedagem em Vercel e Render facilita a publicação da aplicação, permitindo que o cliente acesse o sistema pela web sem necessidade de instalação local.

Assim, o SGI segue uma arquitetura web em camadas, composta por um frontend React hospedado na Vercel, uma API Spring Boot hospedada no Render com suporte a Docker e uma camada de dados persistente. Essa estrutura é adequada ao escopo do projeto, pois centraliza as informações, simplifica o acesso dos usuários e permite futuras evoluções, como integração com serviços externos, melhorias na autenticação e ampliação dos relatórios gerenciais.

.

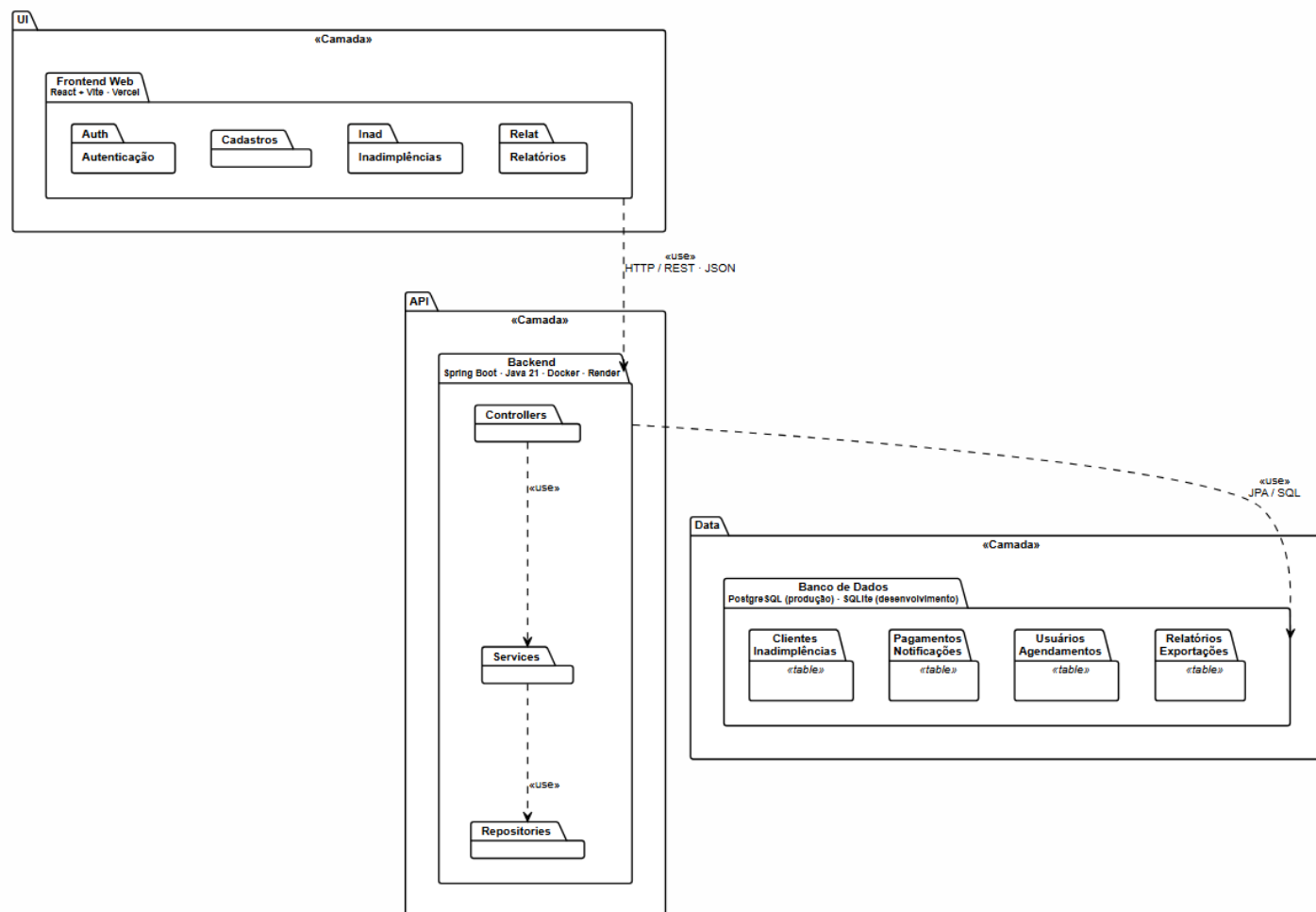


Figura 18 — Diagrama de pacote

3.5 Diagramas de Estados

A Figura 19 apresenta o diagrama de estados da entidade Dívida no SGI — Sistema de Gerenciamento de Inadimplentes. O fluxo inicia com o registro de uma dívida, que pode entrar no estado EM_ABERTO quando possui vencimento futuro ou no estado VENCIDA quando o vencimento já passou ou é identificado pelo serviço agendado do sistema.

A partir desses estados, a dívida pode receber pagamentos parciais, passando para o estado PARCIAL, ou pagamentos totais, passando para o estado QUITADA. Quando uma dívida parcial tem seu saldo restante quitado, ela também é finalizada como QUITADA. O diagrama ainda contempla o estado CANCELADA, utilizado em situações de cancelamento manual da dívida.

Dessa forma, o diagrama representa o ciclo de vida de uma dívida dentro do sistema, demonstrando as principais mudanças de status conforme vencimento, pagamentos e cancelamentos.

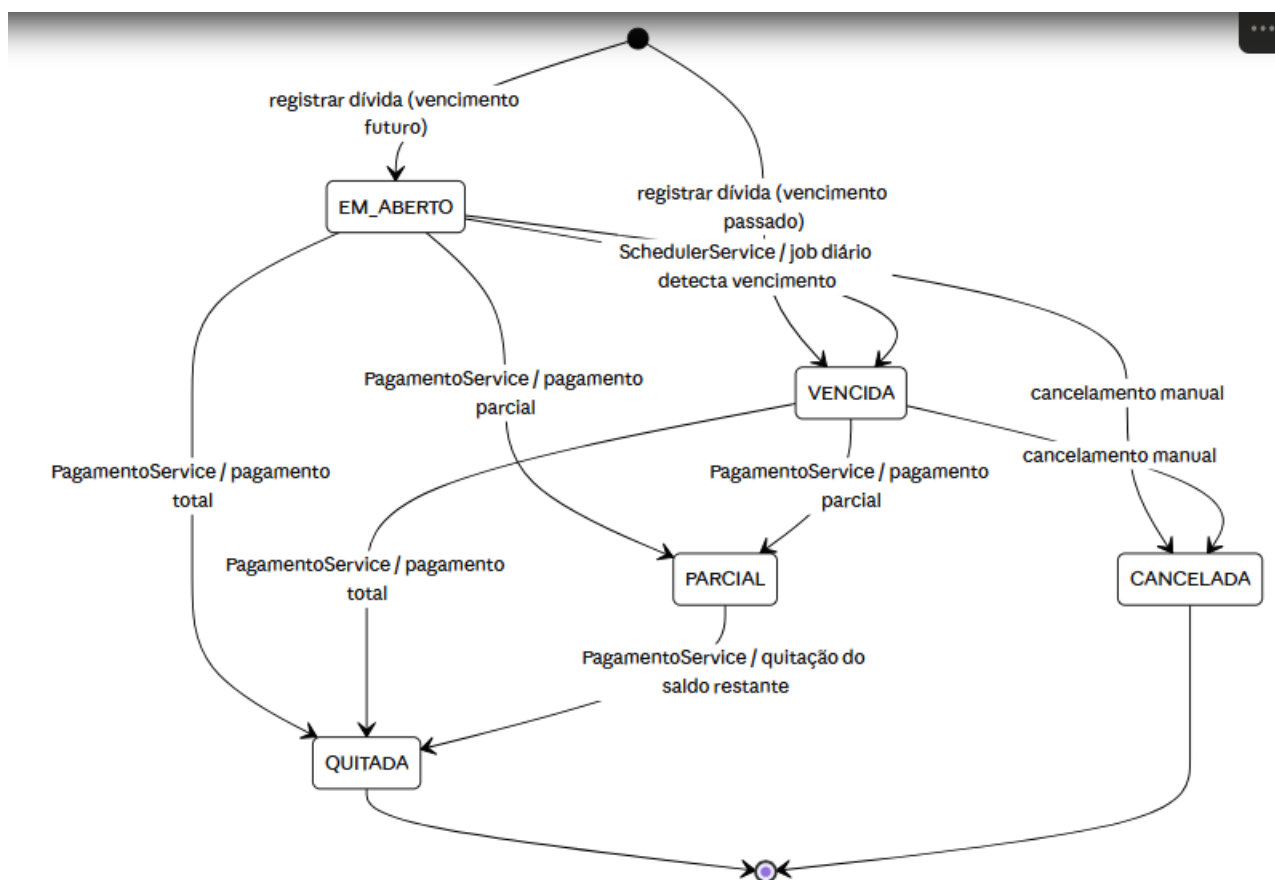


Figura 19 — Diagrama de Estado.

A Figura 20 apresenta o diagrama de estados da notificação de cobrança por e-mail no SGI. O fluxo inicia quando uma notificação de cobrança é criada, entrando inicialmente no estado PENDENTE, aguardando o envio pelo serviço de e-mail.

Quando o EmailService realiza o envio SMTP com sucesso, a notificação passa para o estado ENVIADO, indicando que a comunicação foi entregue ao serviço de envio. Caso ocorra erro durante o envio, a notificação passa para o estado FALHOU. Nesse caso, o sistema pode realizar novas

tentativas por meio de um RetryJob, incrementando o número de tentativas a cada falha.

Se uma nova tentativa for bem-sucedida, a notificação muda para o estado ENVIADO. Dessa forma, o diagrama representa o ciclo de vida de uma notificação de cobrança, contemplando os cenários de envio bem-sucedido, falha e reenvio automático.

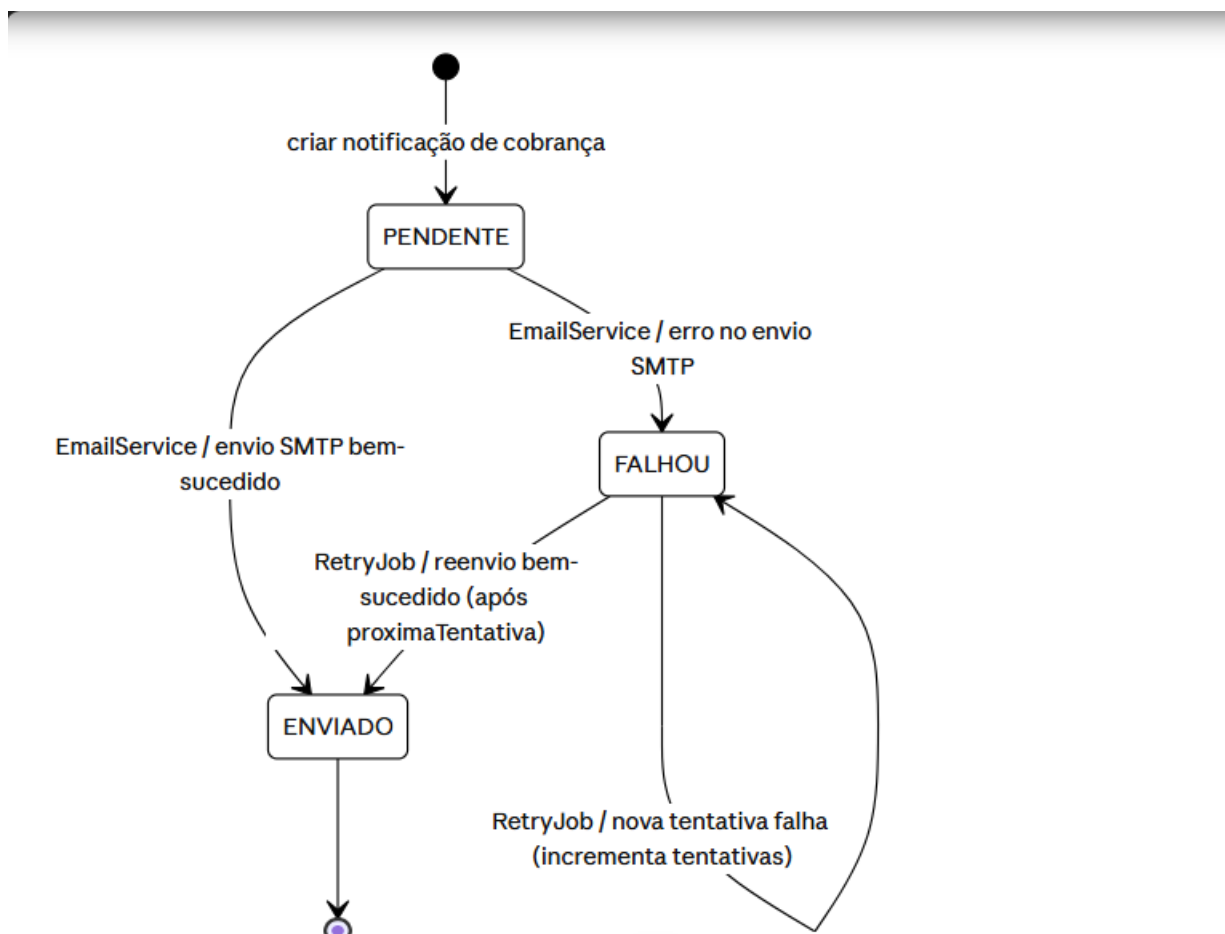


Figura 20 — Diagrama de Estado.

A Figura 21 apresenta o diagrama de estados da entidade Usuário no SGI — Sistema de Gerenciamento de Inadimplentes. O fluxo representa as possíveis situações de acesso de um usuário dentro do sistema, desde sua criação até a ativação ou inativação.

O usuário pode ser criado diretamente pela proprietária, entrando no estado ATIVO, ou pode realizar um cadastro público, ficando inicialmente no estado PENDENTE_APROVACAO. Nesse caso, o acesso ao sistema só é liberado após a aprovação pela proprietária, momento em que o usuário passa para o estado ATIVO.

O diagrama também contempla a revogação de acesso, em que um usuário ativo pode ser inativado por meio de um processo de soft delete, passando para o estado INATIVO. Caso necessário, a

proprietária também pode reativar esse usuário, retornando-o ao estado ATIVO. Dessa forma, o modelo representa o ciclo de vida de acesso dos usuários e contribui para o controle de segurança do sistema.

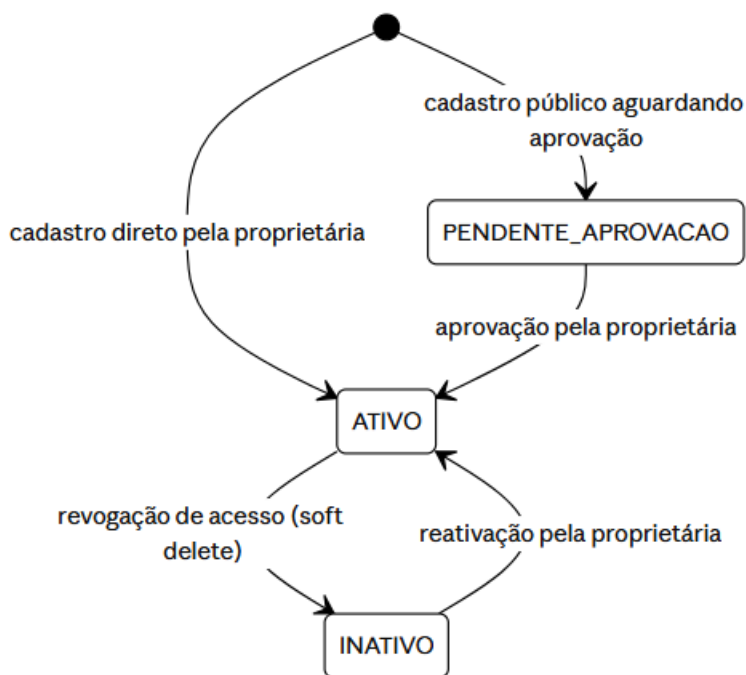


Figura 21 — Diagrama de Estado.

A Figura 22 apresenta o diagrama de estados da entidade Cliente no SGI — Sistema de Gerenciamento de Inadimplentes. O fluxo representa as possíveis situações de um cliente cadastrado no sistema, considerando sua ativação, inativação e posterior reativação.

O ciclo inicia com o cadastro do cliente, momento em que ele passa para o estado ATIVO, permitindo que sejam registradas dívidas, pagamentos e demais informações relacionadas. Caso seja necessário remover o cliente da utilização ativa do sistema, é realizada uma exclusão lógica, também chamada de soft delete, alterando seu estado para INATIVO sem apagar definitivamente seus dados do banco.

O diagrama também contempla a possibilidade de reativação do cadastro, permitindo que um cliente inativo volte ao estado ATIVO. Dessa forma, o modelo preserva o histórico das informações e garante maior controle sobre os registros de clientes no sistema.

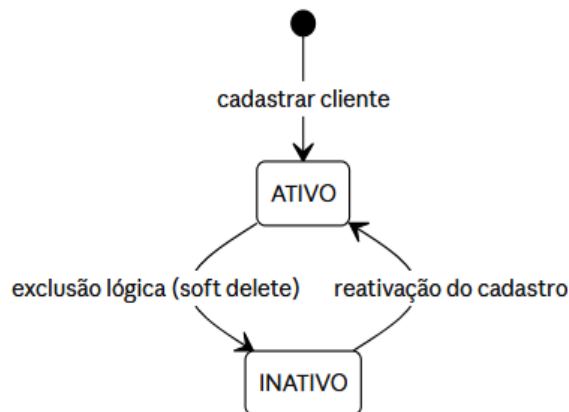


Figura 22 — Diagrama de Estado.

3.6 Diagrama de Componentes e Implantação

O diagrama de componentes apresenta a decomposição lógica do SGI — Sistema de Gerenciamento de Inadimplentes em elementos independentes e suas respectivas interfaces. A solução é organizada em três blocos principais: UI, API e Persistência, seguindo uma arquitetura web em camadas.

Na camada de UI, o sistema possui um Frontend Web desenvolvido com React + Vite, hospedado na plataforma Vercel. Essa camada é executada no navegador do usuário e é responsável pela apresentação das telas, captura das interações e comunicação com a API. Por meio da interface web, os usuários poderão realizar operações como autenticação, cadastro de clientes, registro de inadimplências, registro de pagamentos, consultas, visualização de dashboard e geração de relatórios.

A comunicação entre a camada de UI e a camada de API ocorre exclusivamente por meio de requisições HTTP/HTTPS, utilizando o padrão REST e dados no formato JSON. Dessa forma, o frontend permanece desacoplado do backend, consumindo apenas os endpoints disponibilizados pela API.

A camada de API é implementada com Spring Boot e Java 21, sendo hospedada na plataforma Render. Essa camada expõe os endpoints REST por meio de Controllers, como ClienteController, InadimplenciaController, PagamentoController e RelatorioController. Os Controllers recebem as requisições enviadas pelo frontend e delegam o processamento para os Services, responsáveis por centralizar as regras de negócio do sistema.

Os Services, como ClienteService, InadimplenciaService, PagamentoService e RelatorioService, executam validações e fluxos principais da aplicação, como cadastro de clientes, lançamento de inadimplências, registro de pagamentos, atualização de saldo devedor, alteração do status da inadimplência e consolidação de dados para relatórios. O acesso aos dados é encapsulado por Repositories, utilizando Spring Data JPA, enquanto as Entities/Models representam as entidades persistidas no banco de dados.

Além dos componentes principais, a API também possui componentes transversais de configuração, como políticas de CORS, validação de dados, autenticação/autorização e documentação dos endpoints por meio de ferramentas como OpenAPI/Swagger, quando aplicável. Também podem ser utilizados DTOs e Mappers para padronizar os dados de entrada e saída, evitando o acoplamento direto entre a interface externa da API e as entidades internas do domínio.

Para a implantação do backend, será utilizado Docker, permitindo empacotar a aplicação Spring Boot e suas dependências em um ambiente padronizado. O uso do Docker facilita o deploy no Render, reduz problemas relacionados à configuração de ambiente e contribui para maior previsibilidade entre os ambientes de desenvolvimento e produção.

Na camada de Persistência, o sistema utiliza um banco de dados acessado pela API hospedada no Render. Esse banco é responsável por armazenar informações como usuários, clientes, inadimplências, pagamentos, relatórios e demais registros necessários ao funcionamento do SGI. Em ambiente de desenvolvimento, pode ser utilizado SQLite pela sua simplicidade de configuração. Entretanto, para o ambiente hospedado, recomenda-se o uso de um banco persistente, como PostgreSQL, ou uma configuração com disco persistente no Render, garantindo maior segurança e disponibilidade dos dados.

No diagrama de implantação, o usuário acessa o sistema por meio de um navegador. O navegador carrega o frontend hospedado na Vercel. Quando o usuário executa uma ação, como registrar um pagamento ou consultar clientes inadimplentes, o frontend envia uma requisição para a API hospedada no Render. A API processa a regra de negócio correspondente, acessa a camada de persistência e retorna uma resposta ao frontend.

Essa organização favorece alta coesão e baixo acoplamento entre os componentes, uma vez que cada camada possui responsabilidades bem definidas. A separação entre frontend, backend e banco de dados simplifica a manutenção, facilita testes, melhora a escalabilidade e permite futuras evoluções do sistema sem comprometer a estrutura principal da aplicação.

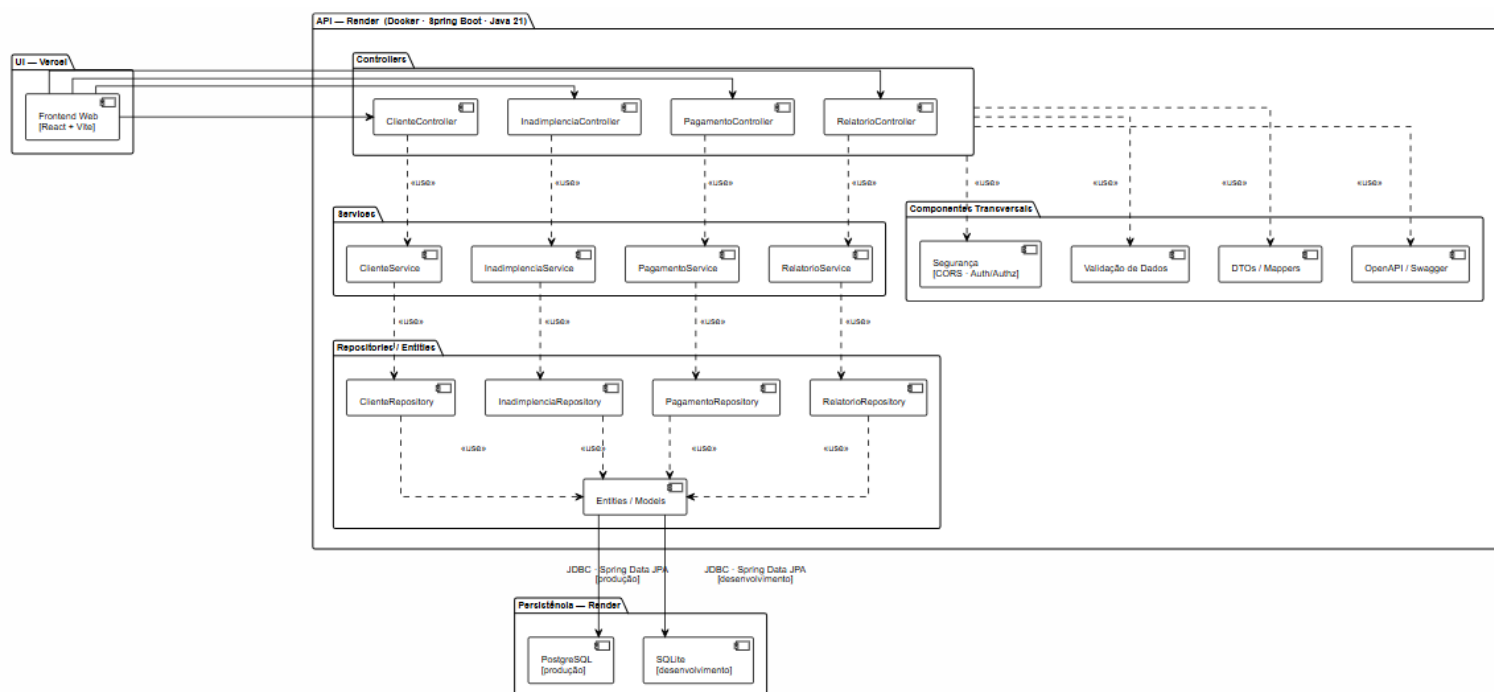


Figura 23 — Diagrama de Componentes

O diagrama de implantação descreve a distribuição física e virtual dos artefatos do SGI — Sistema de Gerenciamento de Inadimplentes. A solução será disponibilizada em ambiente web, dispensando instalação local na máquina do cliente.

O usuário acessa o sistema por meio de um navegador, que carrega o frontend desenvolvido em React + Vite e hospedado na Vercel. Essa camada é responsável pela interface do sistema e pela comunicação com o backend por meio de requisições HTTPS/JSON.

A API do SGI será implementada com Spring Boot e Java 21, hospedada no Render e empacotada com Docker. Essa API expõe os endpoints REST responsáveis por autenticação, cadastro de clientes, registro de inadimplências, registro de pagamentos, consultas, relatórios e atualização de saldo devedor.

A camada de persistência será composta por um banco de dados acessado pela API no ambiente hospedado. Para produção, recomenda-se o uso de um banco persistente, como PostgreSQL, garantindo maior segurança e preservação dos dados.

Assim, a implantação do sistema é composta por três elementos principais: navegador do usuário, frontend na Vercel e backend no Render com banco de dados persistente. Essa organização centraliza os dados, facilita a manutenção e permite que o sistema seja acessado pela web sem instalação local.

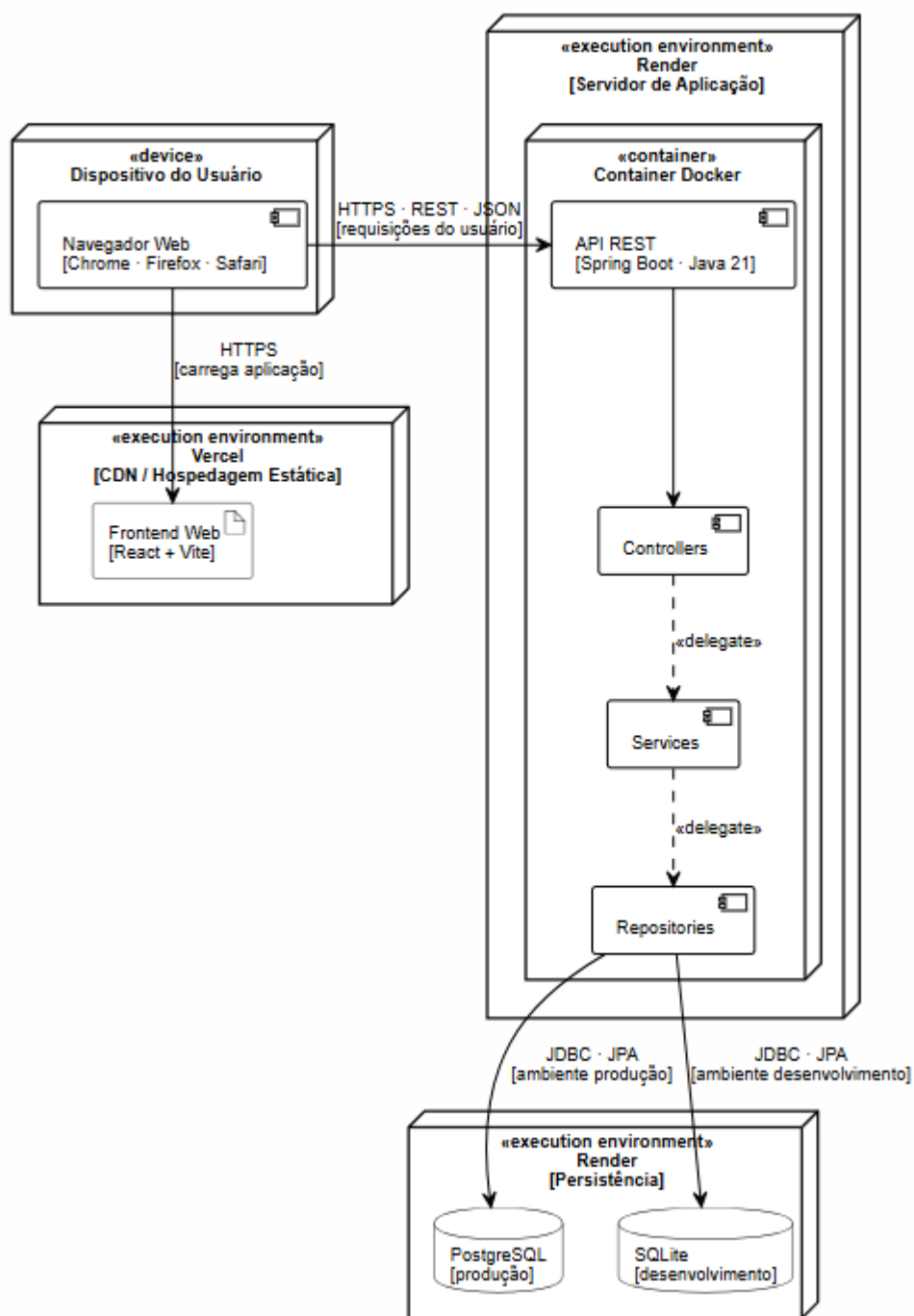


Figura 24 — Diagrama de Implantação

4. Projeto de Interface com Usuário

Esta seção tem como objetivo apresentar e descrever as interfaces de interação com o usuário que compõem a aplicação. Para isso, foram elaborados mockups de baixa fidelidade utilizando a ferramenta Paint.

4.1 Esboço das Interfaces Comuns a Todos os Atores

A seguir, apresenta-se a tela de login do sistema.

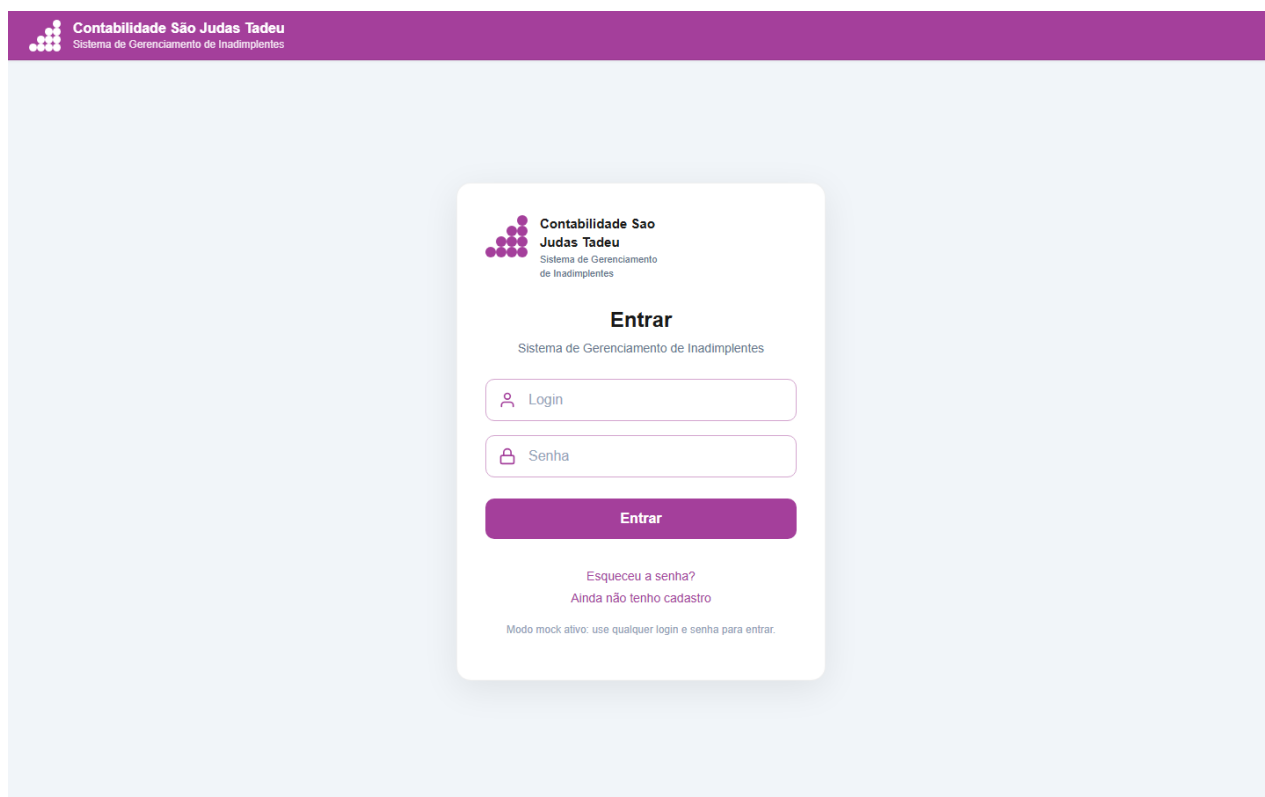


Figura 25 — Tela de Login.

A tela Dashboard apresenta um resumo geral do sistema, exibindo informações como o total de clientes cadastrados, o número de inadimplentes e o valor total em aberto. Além disso, disponibiliza atalhos para o cadastro de clientes e para a área de relatórios. Tanto a Cláudia quanto o José Carlos terão acesso a essa interface, permitindo que acompanhem rapidamente a situação da inadimplência e naveguem para as demais funcionalidades do sistema.



Figura 26. Tela de Dashboard

4.2 Esboço das Interfaces Usada por Ambos

A tela de cadastro de cliente permite o registro de novos clientes no sistema, contendo campos para inserir informações essenciais como nome, CPF/CNPJ, endereço, telefone e e-mail. Após o preenchimento, é possível salvar os dados ou cancelar a operação. Tanto a Cláudia quanto o José Carlos terão acesso a essa interface, garantindo que novos clientes sejam adicionados de forma organizada e padronizada.

Figura 27— Tela de Cadastro de Cliente.

4.3 Esboço das Interfaces Usadas por ambos

A tela de listagem de clientes apresenta uma visão organizada de todos os clientes cadastrados no sistema, exibindo informações como nome, CPF/CNPJ, situação (adimplente ou inadimplente) e telefone para contato.

Contabilidade São Judas Tadeu
Sistema de Gerenciamento de Inadimplentes

Josecarlos

Dashboard

Clientes

Inadimplentes

Serviços

Simulador

Relatórios

Sair

Listagem de Clientes

Gerar relatório Novo Cliente

Buscar clientes... Todos

Nome	CPF/CNPJ	Telefone fixo	Celular	E-mail	Ação
DUVAL DE FIGUEIREDO PIRES	03.536.558/0001-84	—	—	pirespousada@gmail.com	
EDMILSON DE OLIVEIRA TOMAZ E CIA LTDA	03.591.272/0001-00	3868-1995	—	—	
DISTRIBUIDORA DE GAS CASTELAR LTDA EPP	06.350.632/0001-61	3868-1633	—	—	
HELIO TAVARES DE MOURA	01.055.914/0001-12	3868-1066	—	—	
RAIMUNDO QUEIROZ DE ARAUJO	02.416.472/0001-55	3868-1330	—	—	
BJ-RESTAURANTE-COMERCIO & SERVICOS	01.550.700/0001-12	3868-1338	—	bjescapeamentoelanchonete@hotmail.com	
CMDENTRO CARTORIO DE REGISTRO	21.141.148/0001-				

Figura 28 — Tela de Listagem de Clientes.

4.4 Esboço das Interfaces Usadas por ambos

A tela de lista de inadimplentes exibe os clientes que possuem pendências financeiras no sistema. São apresentadas informações como o nome do cliente, o valor da dívida e a data de vencimento, permitindo o acompanhamento da situação de cada inadimplência. Tanto a Cláudia quanto o José Carlos terão acesso a essa interface, de forma a facilitar o controle e a tomada de medidas para regularização dos débitos.

Contabilidade Sao Judas Tadeu
Sistema de Gerenciamento de Inadimplentes

josecarlos

Dashboard

Clientes

Inadimplentes

Serviços

Simulador

Relatórios

Sair

Sistema de Gestão de Inadimplentes

Lista de Inadimplentes

Clientes com pagamentos em atraso

+ Registrar inadimplência

⚙️ Ajustar juros

Total de inadimplentes ⓘ
2

Valor total em aberto
R\$ 7.281,42

Média de atraso
100 dias

Clientes Inadimplentes

Cliente ⓘ	Valor ⓘ	Dias em atraso ⓘ	Ações
Matheus Pereira	R\$ 6.138,46	197 dias	⦿ Ver honorários
DUVAL DE FIGUEIREDO PIRES	R\$ 1.142,96	76 dias	⦿ Ver honorários

Figura 29 — Tela de Lista de Inadimplentes.

A seguir, apresenta-se a tela em que o usuário pode registrar uma nova inadimplência de um cliente.

Contabilidade Sao Judas Tadeu
Sistema de Gerenciamento de Inadimplentes

josecarlos

Dashboard

Clientes

Inadimplentes

Serviços

Simulador

Relatórios

Sair

Sistema de Gestão de Inadimplentes

Lista de Inadimplentes

Clientes com pagamentos em atraso

+ Registrar inadimplência

⚙️ Ajustar juros

Total de inadimplentes ⓘ
2

Valor total em aberto
R\$ 7.281,42

Média de atraso
100 dias

Clientes Inadimplentes

Cliente ⓘ	Valor ⓘ	Dias em atraso ⓘ	Ações
Matheus Pereira	R\$ 6.138,46	197 dias	⦿ Ver honorários
DUVAL DE FIGUEIREDO PIRES	R\$ 1.142,96	76 dias	⦿ Ver honorários

Registro de Inadimplência

Cliente *

Digite para buscar o cliente...

Honorários em aberto

+ Adicionar honorário

Junho 2026 Ex: 150 ou 150.50

Total que o cliente deve: **R\$ 0,00**

Descrição da dívida

Descreva os detalhes da inadimplência (opcional)

Cancelar Salvar

Figura 30 — Tela de Registro de Inadimplência.

Na tela de detalhes das inadimplências, o usuário pode verificar todos os meses em que um cliente possui débitos em aberto. Nessa interface, também é possível confirmar pagamentos, enviar cobranças por e-mail ou WhatsApp e cancelar algum débito, quando necessário.

Contabilidade Sao Judas Tadeu
Sistema de Gerenciamento de Inadimplentes

Proprietária

Dashboard
Clientes
Inadimplentes
Serviços
Simulador
Relatórios
Usuários

[← Voltar para inadimplentes](#)

Sistema de Gestão de Inadimplentes

Honorários em aberto

Comércio Silva Ltda

12.345.678/0001-90
contato@silvasupermercado.com.br

Honorários em aberto	Total em aberto	Maior atraso
2	R\$ 4.050,00	212 dias

Detalhamento por período

Período	Vencimento	Original	Juros	Total
11/2025	15/11/2025	R\$ 2.500,00	R\$ 350,00	R\$ 2.850,00
12/2025	01/12/2025	R\$ 1.200,00	R\$ 0,00	R\$ 1.200,00

PAGAMENTOS REGISTRADOS
Nenhum pagamento parcial registrado.

Total geral em aberto: R\$ 4.050,00

Figura 31 — Tela de Detalhes das Inadimplências.

A tela de serviços permite registrar os serviços prestados e cadastrar novos serviços no sistema.

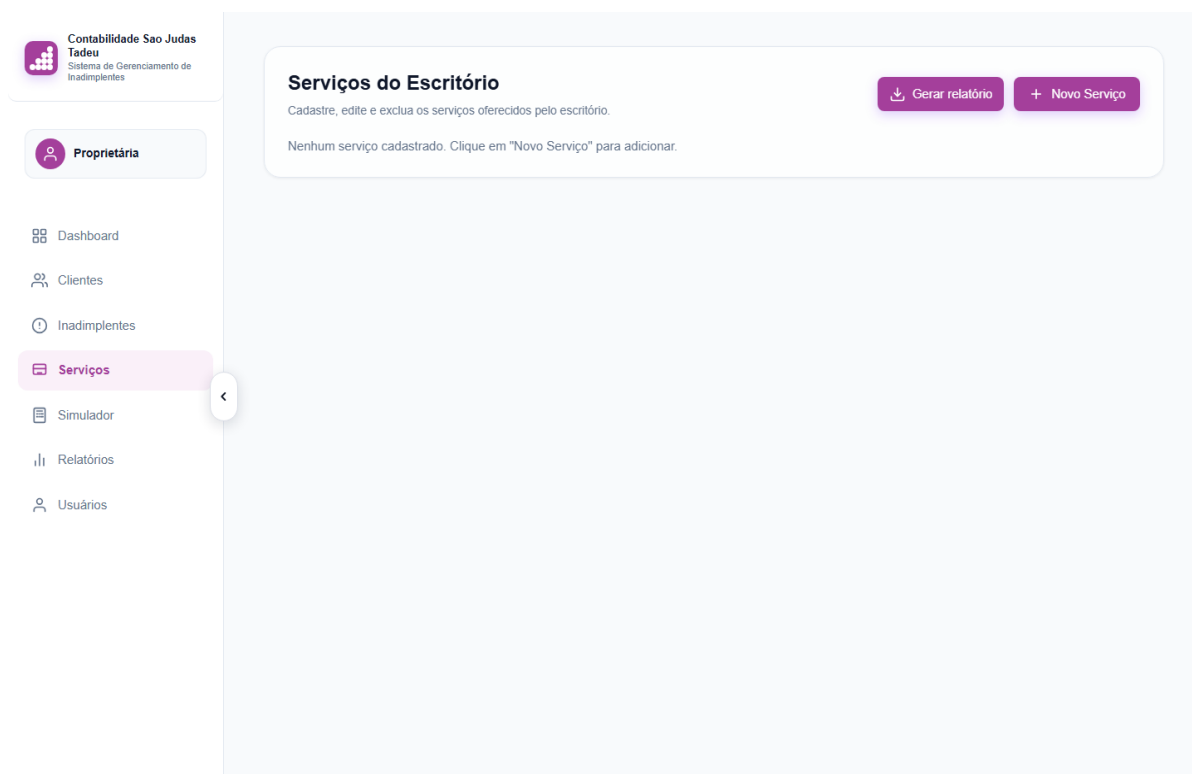


Figura 32 — Tela de Serviços.

A tela de simulador permite ao usuário esclarecer dúvidas e realizar cálculos com base nas novas regras da reforma tributária em vigor no Brasil.

Figura 33 — Tela do Simulador.

Dentro da aba de simulador, há também um chatbot integrado à API do Gemini, por meio do qual o usuário pode tirar dúvidas pontuais, realizar cálculos e fazer pesquisas.



Figura 34 — Chatbot do Simulador.

4.5 Esboço das Interfaces Usadas por ambos

A tela de relatórios possibilita a geração de relatórios personalizados a partir de filtros como período, cliente e status das dívidas. Além da listagem, a interface apresenta informações em formato gráfico, facilitando a análise visual dos dados. Tanto a Cláudia quanto o José Carlos terão acesso a essa funcionalidade, permitindo uma visão estratégica sobre a inadimplência e apoiando o processo de tomada de decisão.

Contabilidade Sao Judas Tadeu
Sistema de Gerenciamento de Inadimplentes

josecarlos

Dashboard

Clientes

Inadimplentes

Serviços

Simulador

Relatórios

Sair

localhost:5173/dashboard

Gestão de Inadimplentes

Relatórios

RANKING DEVEDORES EXTRATO POR CLIENTE INADIMPLÊNCIA POR PERÍODO PAGAMENTOS RECEBIDOS AGING EFETIVIDADE COBRANÇA

Período: Último mês Limite: Top 20 Valor mínimo Qtd. dívidas mín. Dias atraso mín.

Ranking de Maiores Devedores

Posição	Cliente	CPF/CNPJ	Valor Devido	Qtd. Dívidas	Dias Atraso (média)	Status
Gerar relatório Exportar Excel						

Figura 35— Tela de Relatórios.

No perfil da proprietária, há também uma funcionalidade que permite aceitar ou recusar novos registros de login, além de revogar acessos de usuários, quando necessário.

Contabilidade Sao Judas Tadeu
Sistema de Gerenciamento de Inadimplentes

Proprietária

Dashboard

Clientes

Inadimplentes

Serviços

Simulador

Relatórios

Usuários

Gestão de usuários

Usuários

CONTAS ATIVAS CADASTROS PENDENTES

Nome	Login	Data de criação	Status	Ação
Carlos Mendes	novo.usuario	20/02/2026	PENDENTE_APROVACAO	Aprovar

Figura 36 — Tela de Aprovação de Acessos.

Nesta interface, a proprietária pode revogar os acessos de usuários do sistema.

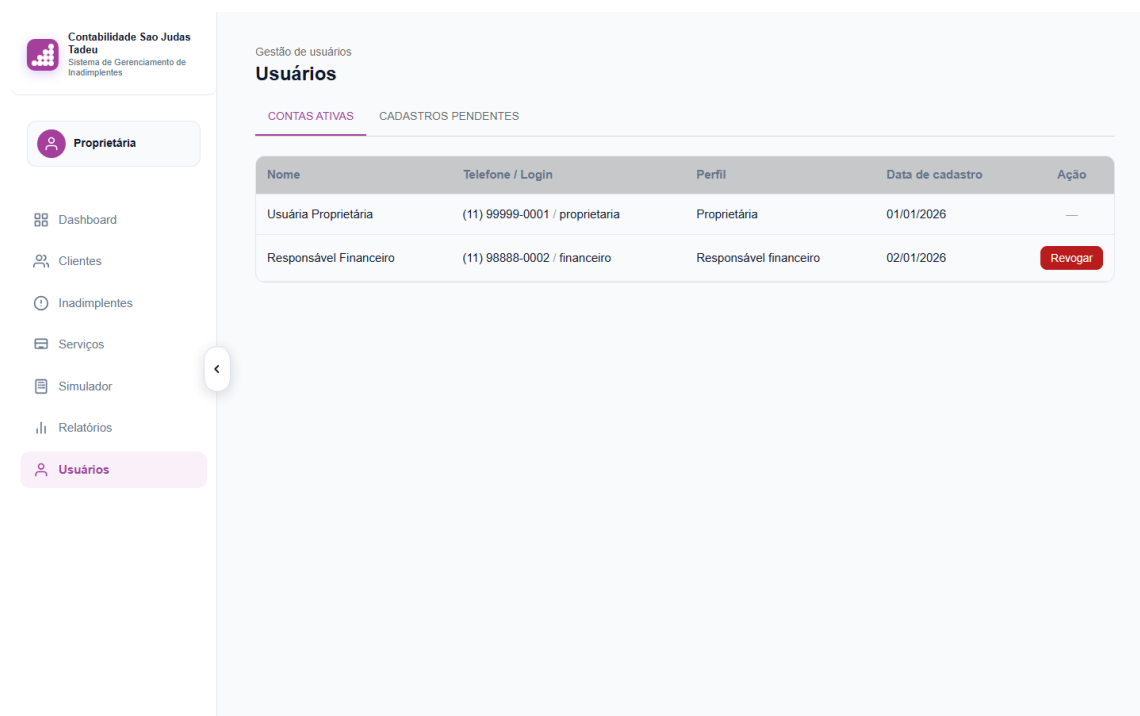


Figura 37 — Tela de Revogação de Acessos.

5. Glossário e Modelos de Dados

Esta seção descreve o glossário do sistema e o modelo de dados do SGI – Sistema de Gerenciamento de Inadimplentes. Primeiro, apresentam-se tabelas com os atributos de entrada e saída usados nas telas e relatórios, padronizando nome, formato e significado. Em seguida, apresenta-se o modelo relacional (DER e DDL essencial) e as estratégias de mapeamento entre objetos da aplicação (camada de domínio) e tabelas do banco de dados.

ACESSO

Atributo	Formato	Descrição
Login	String	Telefone usado no login.
senha	String	Hash/salt da senha (se aplicável).
perfil	enum	Perfil de acesso
tokenacesso	String	Token
statusUsuario	Enum	Situação

Tabela 1 acesso.

A Tabela 1 apresenta o *dicionário de dados* relacionado à autenticação e sessão do sistema, descrevendo os principais campos utilizados para o login e controle de permissões. Ela padroniza o formato dos dados (ex.: *String* e *Enum*) e explicita o significado de cada atributo, incluindo credenciais, perfil do usuário, token de acesso e status, garantindo consistência entre UI, API e banco de dados.

TABELA CLIENTE

Atributo	Formato	Descrição
clienteId	UUID	Identificador Cliente
nome	String	Nome completo.
cpfcnpj	String	Documento do cliente
email	String	email
telefone	String	Telefone para contato
endereço	String	Endereço completo
statuscliente	Enum {Ativo, Inativo}	Situação do cadastro
saldo	INTEGER	Saldo
atualizadoem	Timestamp	Ultima atualização

Tabela 2 Tabela Cliente..

A Tabela 2 (Cliente) documenta os atributos do cadastro de clientes e os campos que sustentam as funcionalidades de consulta, listagem e atualização no SGI. Além dos dados de identificação e contato, a tabela define campos de controle (status do cliente) e dados financeiros (saldo e data de última atualização), servindo como referência para validações, persistência e geração de relatórios

TABELA RELATÓRIO E INDICADORES

Atributo	Formato	Descrição
rankingMaioresDevedores	Lista {clienteId, nome, saldoCentavos}	Ranking ordenado por saldo.
resumoPeriodo	{ini,fim, totalPago, totalAberto}	Indicadores por intervalo.
exportArquivo	PDF/Excel	Resultado exportado sob demanda.

Tabela 3 relatório e indicadores.

A Tabela 3 descreve os dados de saída usados nas telas de relatórios e dashboards, incluindo estruturas agregadas (ex.: rankings) e resumos por período. Ela deixa claro quais campos compõem cada visão gerencial e quais formatos são suportados para exportação (PDF/Excel), garantindo rastreabilidade entre requisitos (US4–US6) e os resultados apresentados ao usuário.

TABELA PAGAMENTO

Atributo	Formato	Descrição
pagamentoId	UUID/API • INTEGER/BD	Identificador do pagamento.
clienteId	FK → Cliente	Cliente vinculado.
valorPago	INTEGER	Valor pago
dataPagamento	Date	Data do pagamento.
comprovante	Blob/URL	Comprovante (opcional).
criadoEm	Timestamp	Registro do lançamento.

Tabela 4 Pagamentos

A Tabela 4 (Pagamento) define o conjunto de informações necessárias para registrar um pagamento no sistema, incluindo identificadores, vínculo com o cliente, valor, data e comprovante (quando existir). Essa tabela também padroniza os campos de auditoria/controle (criadoEm), ajudando a manter histórico e consistência operacional nos fluxos de quitação parcial ou total.

TABELA INADIMPLÊNCIA

Atributo	Formato	Descrição
inadimplenciaId	UUID/API • INTEGER/BD	Identificador da dívida.
clienteId	FK → Cliente	Cliente associado.
valorDevidoCentavos	INTEGER	Valor da dívida (centavos).
vencimento	Date	Data de vencimento.
descricao	String	Observações (opcional).
statusInadimplencia	Enum	Estado da dívida.

criadoEm	Timestamp	Momento do registro.
----------	-----------	----------------------

Tabela 5 Tabela Inadimplência

A Tabela 5 (Inadimplência) apresenta os atributos que representam uma pendência financeira (dívida) associada a um cliente, registrando valor, vencimento, descrição e o status da inadimplência. Ela é a base do acompanhamento de débitos no SGI e sustenta regras de negócio como validação de vencimento, transições de status e cálculo/atualização do saldo apresentado nas telas do sistema.

A Figura 21 abaixo apresenta a modelagem relacional do SGI, evidenciando as entidades persistidas (tabelas) e seus relacionamentos por chaves primárias e estrangeiras. O diagrama consolida a estrutura do banco que suporta o cadastro de clientes, o registro de inadimplências e pagamentos e a geração de visões para relatórios, servindo como referência para implementação do DDL, integridade referencial e mapeamento ORM (JPA/Hibernate)

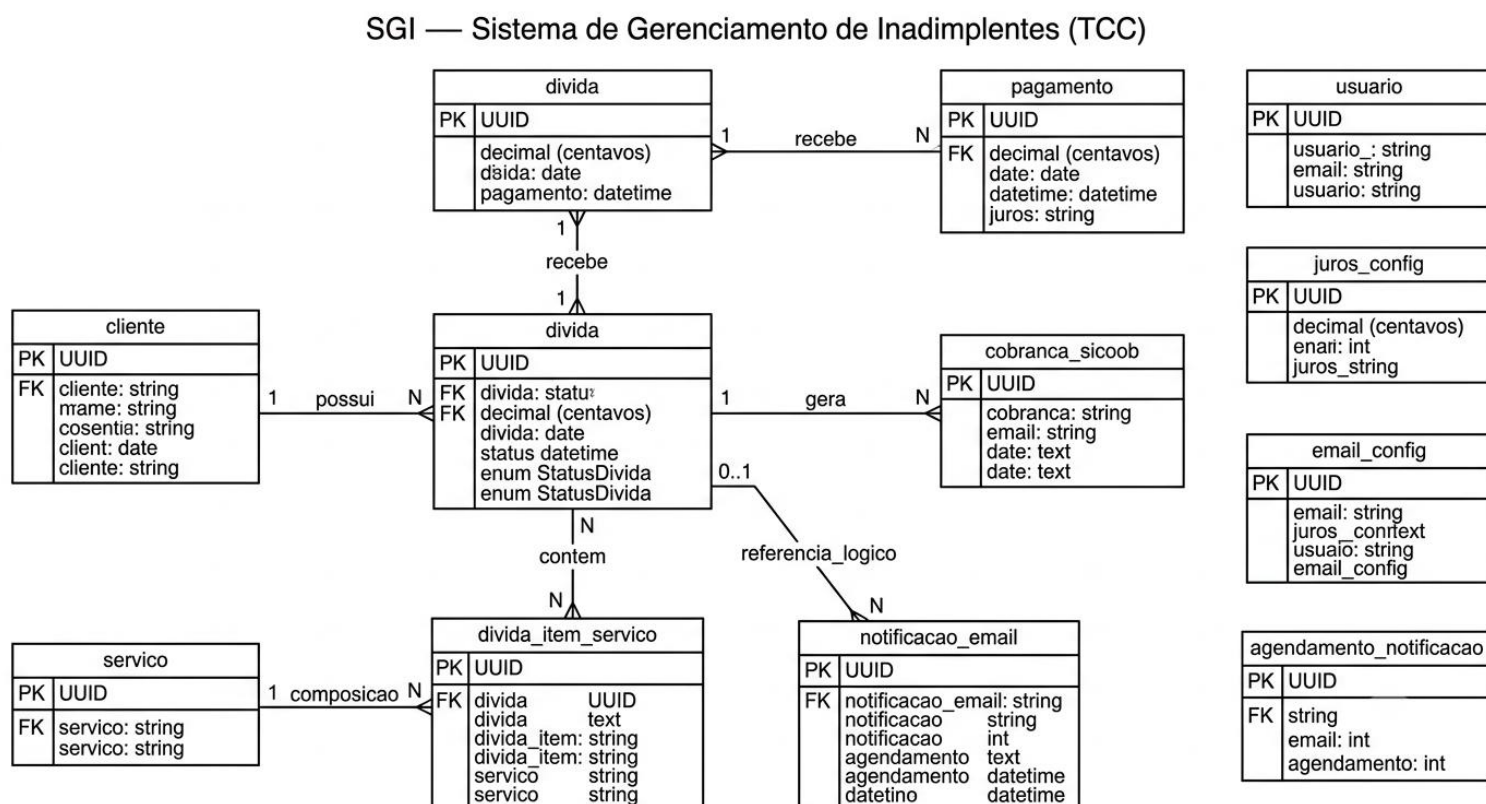


Figura 38—diagrama DER

6. Casos de Teste

O Sistema de Gerenciamento de Inadimplentes (SGI) será validado principalmente por meio de testes funcionais de caixa-preta, executados manualmente em ambiente de homologação com a mesma configuração prevista para produção. Os testes foram derivados dos casos de uso, das histórias de usuário (US1 a US10) e dos contratos de operação, contemplando autenticação, cadastro de clientes, registro de inadimplências, registro de pagamentos, atualização de saldo e geração de relatórios. Além dos testes funcionais, serão observados requisitos não funcionais básicos, como usabilidade (telas claras e coerentes com o trabalho diário do escritório), desempenho adequado nas operações mais comuns e segurança de acesso às informações. O sistema será considerado aceito quando os casos de teste críticos (login, cadastro de cliente, registro de inadimplência, pagamento, recálculo de saldo e relatórios principais) estiverem aprovados e não houver defeitos bloqueadores nas principais funcionalidades.

6.1 Plano de Teste de Aceitação

Os testes de aceitação têm como objetivo verificar se o SGI atende às principais necessidades do escritório de contabilidade, alinhadas ao Documento de Visão e às histórias de usuário. Para fins de organização, foram definidas cinco necessidades principais, e para cada uma delas são apresentados três casos de teste de aceitação.

- **Necessidade 1** – Garantir acesso seguro ao sistema (autenticação e perfis).
- **Necessidade 2** – Manter o cadastro de clientes atualizado e consistente.
- **Necessidade 3** – Registrar inadimplências dos clientes.
- **Necessidade 4** – Registrar pagamentos e atualizar automaticamente o saldo devedor.
- **Necessidade 5** – Disponibilizar relatórios e indicadores para apoio à gestão.

O identificador dos casos de teste segue o padrão T<número do teste>A<número da necessidade>. Exemplo: T1A1 corresponde ao primeiro teste da necessidade 1.

6.1.1 Necessidade 1 – Acesso seguro ao sistema

Caso de Teste T1A1 – Login com credenciais válidas

Campo	Descrição
Identificador	T1A1
Caso de teste	Realizar login com credenciais válidas e acessar o sistema conforme o perfil.

Pré-condições	Usuário cadastrado, ativo no sistema; telefone/senha corretos; aplicação Web ou Desktop em execução.
Ações	(1) Abrir a tela de login. (2) Informar telefone e senha válidos. (3) Acionar o botão "Entrar".
Resultados esperados	A autenticação é realizada com sucesso; o sistema gera um token de sessão; o usuário é redirecionado para o dashboard inicial de acordo com seu perfil.

Tabela 6 Tabela Caso de Teste T1A1.

Caso de Teste T1A1 (Login com credenciais válidas): Esta tabela valida o acesso ao sistema quando o usuário informa credenciais corretas, garantindo autenticação bem-sucedida e redirecionamento conforme o perfil. O aceite ocorre quando o sistema cria a sessão/token e libera o acesso ao dashboard.

Caso de Teste T2A1 – Login com senha inválida

Campo	Descrição
Identificador	T2A1
Caso de teste	Tentar realizar login com senha inválida.
Pré-condições	Usuário cadastrado e ativo; telefone correto; senha incorreta.
Ações	(1) Abrir a tela de login. (2) Informar telefone válido e senha incorreta. (3) Acionar o botão "Entrar".
Resultados esperados	O sistema recusa o acesso, não gera token de sessão e exibe mensagem de erro informando credenciais inválidas, mantendo o usuário na tela de login.

Tabela 7 Tabela Caso de Teste T2A1.

Caso de Teste T2A1 (Login com senha inválida): Esta tabela verifica se o sistema bloqueia corretamente tentativas de login com senha incorreta, sem criar sessão. O aceite ocorre quando o erro é exibido e o usuário permanece na tela de login.

Caso de Teste T3A1 – Acesso sem autenticação a área restrita

Campo	Descrição
Identificador	T3A1
Caso de teste	Tentar acessar diretamente uma tela restrita sem estar autenticado.

Pré-condições	Nenhuma sessão autenticada ativa; URL de tela restrita conhecida (por exemplo, lista de inadimplentes).
Ações	(1) Tentar acessar a URL de uma tela restrita digitando o endereço direto no navegador ou abrindo o app sem login.
Resultados esperados	O sistema impede o acesso e redireciona o usuário para a tela de login, exibindo mensagem informando a necessidade de autenticação.

Tabela 8 Tabela Caso de Teste T3A1.

Caso de Teste T3A1 (Acesso sem autenticação): Esta tabela comprova que áreas restritas não podem ser acessadas sem login, reforçando a segurança do SGI. O aceite ocorre quando o sistema redireciona para o login e informa a necessidade de autenticação.

6.1.2 Necessidade 2 – Cadastro e manutenção de clientes

Caso de Teste T1A2 – Cadastrar novo cliente com dados válidos

Campo	Descrição
Identificador	T1A2
Caso de teste	Cadastrar um novo cliente com todas as informações obrigatórias preenchidas corretamente.
Pré-condições	Usuário autenticado com permissão de cadastro; conexão com o banco de dados ativa.
Ações	(1) Acessar a tela "Cadastro de Cliente". (2) Preencher nome, CPF/CNPJ, endereço, telefone e e-mail válidos. (3) Acionar o botão "Salvar".
Resultados esperados	O sistema valida os dados, salva o novo cliente na base, exibe mensagem de sucesso e o cliente passa a aparecer na lista de clientes cadastrados

Tabela 9 Tabela Caso de Teste T1A2

Esta tabela valida o cadastro completo de um cliente com dados obrigatórios corretos, garantindo persistência e retorno de sucesso. O aceite ocorre quando o cliente passa a aparecer na listagem após o salvamento.

Caso de Teste T2A2 – Impedir cadastro com CPF/CNPJ duplicado

Campo	Descrição
Identificador	T2A2
Caso de teste	Impedir o cadastro de um cliente com CPF/CNPJ já existente.
Pré-condições	Cliente já cadastrado com determinado CPF/CNPJ; usuário autenticado com permissão de cadastro.
Ações	(1) Acessar a tela "Cadastro de Cliente". (2) Informar um CPF/CNPJ já existente. (3) Preencher demais campos e acionar "Salvar".
Resultados esperados	O sistema identifica o CPF/CNPJ duplicado, não grava o cadastro e exibe mensagem de erro informando que já existe cliente com esse documento.

Tabela 10 Tabela Caso de Teste T2A2

Esta tabela verifica a regra de unicidade de documento, impedindo o cadastro de clientes duplicados. O aceite ocorre quando o sistema bloqueia o salvamento e exibe a mensagem de erro adequada.

Caso de Teste T3A2 – Atualizar dados de cliente existente

Campo	Descrição
Identificador	T3A2
Caso de teste	Editar os dados de um cliente já cadastrado.
Pré-condições	Cliente existente na base; usuário autenticado.
Ações	(1) Acessar a tela "Listagem de Clientes". (2) Selecionar um cliente. (3) Acionar a opção "Editar". (4) Alterar endereço/telefone. (5) Salvar alterações.
Resultados esperados	As alterações são validadas e salvas; o sistema exibe mensagem de sucesso; a lista de clientes e os detalhes do cliente são atualizados com os novos dados.

Tabela 11 Tabela Caso de Teste T3A2.

Esta tabela valida a edição de dados de um cliente existente, garantindo que alterações sejam persistidas e refletidas na interface. O aceite ocorre quando o sistema confirma sucesso e atualiza os dados exibidos.

6.1.3 Necessidade 3 – Registro de inadimplências

Caso de Teste T1A3 – Registrar nova

inadimplência

Campo	Descrição
Identificador	T1A3
Caso de teste	Registrar uma nova inadimplência para um cliente ativo.
Pré-condições	Cliente cadastrado e ativo; usuário autenticado; tela de registro de inadimplência acessível.
Ações	(1) Acessar a tela "Registro de Inadimplência". (2) Selecionar o cliente. (3) Informar valor devido, data de vencimento e descrição. (4) Acionar "Salvar".
Resultados esperados	A inadimplência é validada e registrada com status inicial "Em Aberto"; o sistema gera um identificador/protocolo e exibe mensagem de sucesso.

Tabela 12 Tabela Caso de Teste T1A3.

Esta tabela valida o registro de uma dívida para um cliente ativo, garantindo validação, criação do lançamento e status inicial apropriado. O aceite ocorre quando o sistema confirma o registro e gera um identificador/protocolo.

Caso de Teste T2A3 – Impedir registro com dados inválidos

Campo	Descrição
Identificador	T2A3
Caso de teste	Impedir o registro de inadimplência com valor inválido ou cliente inexistente.
Pré-condições	Usuário autenticado.

Ações	(1) Acessar "Registro de Inadimplência". (2) Informar valor ≤ 0 ou selecionar um cliente inexistente (ou removido). (3) Acionar "Salvar".
Resultados esperados	O sistema valida os campos, não grava a inadimplência e exibe mensagens de erro indicando o problema (valor inválido ou cliente não encontrado).

Tabela 13 Tabela Caso de Teste T2A3

Esta tabela verifica as validações do registro de inadimplência (ex.: valor inválido ou cliente inexistente), evitando dados inconsistentes. O aceite ocorre quando o sistema não grava a informação e apresenta mensagens claras de erro.

Caso de Teste T3A3 – Listar inadimplências por cliente

Campo	Descrição
Identificador	T3A3
Caso de teste	Exibir a lista de inadimplências filtrada por cliente.
Pré-condições	Cliente com uma ou mais inadimplências registradas; usuário autenticado.
Ações	(1) Acessar a tela "Lista de Inadimplentes". (2) Aplicar filtro pelo cliente selecionado.
Resultados esperados	O sistema apresenta apenas as inadimplências vinculadas ao cliente filtrado, exibindo valor, vencimento e status de cada registro.

Tabela 14 Tabela Caso de Teste T3A3.

Esta tabela valida a consulta filtrada das inadimplências de um cliente, garantindo que apenas registros vinculados sejam exibidos. O aceite ocorre quando a lista retorna com valor, vencimento e status corretamente associados ao cliente.

6.1.4 Necessidade 4 – Registro de pagamentos e atualização de saldo

Caso de Teste T1A4 – Registrar pagamento parcial e recalcular

Esta tabela valida o registro de pagamento parcial e o recálculo do saldo, garantindo atualização do status da dívida (ex.: “Parcial”) e exibição do novo saldo. O aceite ocorre quando o pagamento é persistido e o saldo permanece em aberto corretamente.

Campo	Descrição
Identificador	T1A4
Caso de teste	Registrar um pagamento parcial para uma dívida em aberto, recalculando o saldo devedor.

Pré-condições	Cliente com inadimplência em status "Em Aberto"; usuário autenticado; valor devido maior que o valor a pagar.
Ações	(1) Acessar tela de "Registro de Pagamento". (2) Selecionar o cliente e a dívida. (3) Informar valor de pagamento menor que o saldo total. (4) Confirmar operação.
Resultados esperados	O pagamento é registrado; o saldo da dívida é recalculado e permanece em aberto; o status da dívida é atualizado (por exemplo, "Parcial"); o novo saldo é exibido ao usuário.

Tabela 15 Tabela Caso de Teste T1A4.

Caso de Teste T2A4 – Quitar dívida e atualizar status do cliente

Campo	Descrição
Identificador	T2A4
Caso de teste	Registrar pagamento que quita a dívida e atualizar o status do cliente.
Pré-condições	Cliente com uma única dívida em aberto; usuário autenticado; valor de pagamento igual ao saldo devedor.
Ações	(1) Acessar "Registro de Pagamento". (2) Selecionar o cliente e a dívida. (3) Informar valor igual ao saldo devedor. (4) Confirmar pagamento.
Resultados esperados	A dívida é marcada como quitada; o saldo é recalculado para zero; o status do cliente é atualizado para "Sem Débito/Adimplente"; recibo/registro é exibido.

Tabela 16 Tabela Caso de Teste T2A4.

Esta tabela verifica o fluxo de quitação total, garantindo saldo zerado, marcação de dívida como quitada e atualização do status do cliente. O aceite ocorre quando o sistema registra o pagamento e apresenta a confirmação/recibo com o novo estado.

Caso de Teste T3A4 – Impedir pagamento inválido

Esta tabela abaixo valida as regras de bloqueio para pagamentos inconsistentes (ex.: valor maior que saldo ou cliente/dívida inexistentes). O aceite ocorre quando o sistema não registra o pagamento e retorna mensagens de erro específicas.

Campo	Descrição
Identificador	T3A4

Caso de teste	Impedir o registro de pagamento com valor maior que o saldo ou para cliente inexistente.
Pré-condições	Usuário autenticado.
Ações	(1) Acessar "Registro de Pagamento". (2) Selecionar cliente/dívida ou informar dados incorretos. (3) Informar valor maior que o saldo ou apontar cliente inexistente.
Resultados esperados	O sistema valida os dados, não registra o pagamento e apresenta mensagens de erro adequadas (valor maior que o saldo, cliente/dívida não encontrados).

Tabela 17 Tabela Caso de Teste T3A4.

6.1.5 Necessidade 5 – Relatórios e indicadores

Caso de Teste T1A5 – Gerar relatório de inadimplentes por período

Esta tabela valida a geração de relatório de inadimplentes com filtro de período, garantindo consistência dos dados exibidos (valores, vencimentos e total em aberto). O aceite ocorre quando o sistema retorna o relatório apenas com registros dentro do intervalo informado.

Campo	Descrição
Identificador	T1A5
Caso de teste	Gerar relatório de inadimplentes filtrado por período de vencimento.
Pré-condições	Usuário autenticado; existência de inadimplências registradas em diferentes períodos
Ações	(1) Acessar a tela "Relatórios". (2) Informar período inicial e final. (3) Acionar a geração do relatório.
Resultados esperados	O sistema consulta os dados e exibe relatório com clientes inadimplentes dentro do período, com informações de valor, vencimento e total em aberto.

Tabela 17 Tabela Caso de Teste T1A5.

Caso de Teste T2A5 – Exportar relatório em PDF/Excel

Esta tabela verifica a exportação do relatório gerado, garantindo que o arquivo final mantenha os mesmos dados apresentados na tela. O aceite ocorre quando o sistema disponibiliza o download no formato escolhido (PDF ou Excel).

Campo	Descrição
Identificador	T2A5
Caso de teste	Exportar um relatório gerado para PDF ou Excel.

Pré-condições	Relatório já gerado na tela; usuário autenticado.
Ações	(1) Na tela de relatórios, após gerar o relatório, acionar a opção "Exportar". (2) Selecionar o formato (PDF ou Excel).
Resultados esperados	O sistema gera o arquivo no formato escolhido e disponibiliza para download; o arquivo contém os mesmos dados apresentados na tela.

Tabela 18 Tabela Caso de Teste T2A5.

Caso de Teste T3A5 – Visualizar dashboard com indicadores

Campo	Descrição
Identificador	T3A5
Caso de teste	Visualizar dashboard com indicadores de inadimplência (total em aberto, número de inadimplentes, ranking de maiores devedores).
Pré-condições	Usuário autenticado; existência de clientes e inadimplências no sistema.
Ações	(1) Acessar a tela "Dashboard".
Resultados esperados	O sistema apresenta os indicadores consolidados (totais, gráficos, ranking), atualizados a partir dos dados do banco, permitindo visão rápida da situação financeira.

Tabela 20 Tabela Caso de Teste T3A5.

Esta tabela valida a visualização de indicadores consolidados (totais, gráficos e ranking), garantindo atualização a partir do banco e leitura rápida do cenário financeiro. O aceite ocorre quando os números exibidos refletem corretamente os dados cadastrados no sistema

6.2 Plano de Teste de Integração

Os testes de integração têm como objetivo verificar se os componentes do SGI interagem corretamente entre si e com recursos externos, como a API, a camada de persistência e as diferentes interfaces cliente.

A seguir são apresentados casos de teste de integração que cobrem os fluxos principais: autenticação, cadastro de clientes, registro de inadimplência, registros de pagamentos com recálculo de saldo e geração/exportação de relatórios.

Caso de Teste TI1 – Integração UI ↔ API ↔ Banco de Dados na autenticação

Campo	Descrição
Identificador	TI1
Caso de teste	Verificar integração entre interface (Web/Desktop), API de autenticação e banco de dados ao realizar login.
Pré-condições	Usuário cadastrado e ativo na base (tabela de acesso); API em execução; banco de dados acessível; frontend configurado apontando para a URL da API.
Ações	(1) No frontend, informar telefone e senha válidos. (2) Enviar requisição de login para a API. (3) API consultar repositório de usuários no banco.
Resultados esperados	A API valida as credenciais contra o banco e retorna resposta HTTP 200 com token e perfil; o frontend armazena o token (sessão) e redireciona para o dashboard.

Tabela 21 Tabela Caso de Teste TI1

Caso de Teste TI2 – Integração no cadastro de clientes

Campo	Descrição
Identificador	TI2
Caso de teste	Verificar o fluxo integrado de cadastro de cliente, do frontend até a persistência em banco.
Pré-condições	Usuário autenticado; API em execução; tabela de clientes criada; conexão com o banco de dados funcionando.
Ações	(1) No frontend, abrir a tela "Cadastro de Cliente". (2) Preencher dados válidos. (3) Enviar requisição de cadastro para a API. (4) API chamar o Service/Repository.
Resultados esperados	A API valida os dados e persiste o cliente via Repository; o banco grava o registro; a resposta HTTP de sucesso é enviada ao frontend, que exibe mensagem e atualiza a listagem.

Tabela 22 Tabela Caso de Teste TI2.

Caso de Teste TI3 – Integração no registro de inadimplência

Campo	Descrição
Identificador	TI3
Caso de teste	Verificar a integração entre UI, API e banco no registro de inadimplência vinculada a um cliente
Pré-condições	Cliente existente na base; usuário autenticado; API e banco em execução.
Ações	(1) Na UI, acessar "Registro de Inadimplência". (2) Selecionar cliente e informar valor, vencimento e descrição. (3) Enviar requisição para a API.
Resultados esperados	A API aciona o serviço de inadimplência, valida dados, grava o registro no banco e retorna protocolo/ID; a UI exibe confirmação e a inadimplência passa a aparecer na lista do cliente.

Tabela 23 Tabela Caso de Teste TI3

Caso de Teste TI4 – Integração no registro de pagamento com recálculo de saldo

Campo	Descrição
Identificador	TI4
Caso de teste	Verificar se o registro de pagamento integra corretamente UI, API, serviços de domínio e banco, atualizando o saldo devedor.
Pré-condições	Cliente com inadimplência em aberto; usuário autenticado; API e banco em execução; serviço de recálculo de saldo implementado.
Ações	(1) Na UI, acessar "Registro de Pagamento". (2) Selecionar cliente e inadimplência. (3) Informar valor de pagamento. (4) Enviar requisição à API.
Resultados esperados	A API registra o pagamento, aciona o serviço de recálculo de saldo, atualiza o registro de inadimplência e o saldo do cliente no banco; em seguida retorna novo saldo e status; a UI exibe os valores atualizados.

Tabela 24 Tabela Caso de Teste TI4.

Caso de Teste TI5 – Integração na geração e exportação de relatórios

Campo	Descrição
Identificador	TI5
Caso de teste	Verificar a integração entre UI, API, camada de relatório e banco de dados na geração e exportação de relatórios.
Pré-condições	Dados de clientes, inadimplências e pagamentos já cadastrados; usuário autenticado; API e banco operacionais.
Ações	(1) Na UI, acessar "Relatórios". (2) Informar filtros de período/cliente/status. (3) Enviar requisição para geração de relatório. (4) Após exibição, acionar "Exportar".

Resultados esperados	A API consulta o banco, consolida os dados e retorna dataset para a UI; a UI exibe o relatório; ao exportar, a API gera arquivo PDF/Excel e devolve para download, mantendo consistência com os dados exibidos.
----------------------	---

Tabela 25 Tabela Caso de Teste T15. Cronograma e Processo de Implementação

7. Cronograma

O desenvolvimento do Sistema de Gerenciamento de Inadimplentes (SGI) adotará um processo incremental e iterativo, inspirado em práticas ágeis. As funcionalidades serão organizadas em histórias de usuário e distribuídas em ciclos de trabalho quinzenais, priorizando primeiro o núcleo do sistema (cadastro de clientes, registro de inadimplências, pagamentos e relatórios principais). Cada ciclo envolverá atividades de planejamento (seleção das histórias e definição de critérios de aceitação), implementação (backend em Spring Boot, banco de dados e interfaces), revisão de código e execução de testes. Como artefatos principais, serão produzidos e continuamente atualizados: Documento de Visão, backlog de requisitos, modelos UML (casos de uso, sequência, comunicação e arquitetura), modelo de dados, código-fonte versionado, evidências de testes e versões executáveis do sistema para homologação.

Para apoiar esse processo, será utilizado Git como sistema de controle de versão e um repositório remoto (por exemplo, GitHub) para centralizar o código, a documentação e o histórico de alterações. A gerência de projeto será feita por meio de um quadro kanban (GitHub Projects, Trello ou similar), com tarefas organizadas em colunas como “A Fazer”, “Em andamento” e “Concluído”, permitindo acompanhar o progresso por quinzena. Será seguido um fluxo de branches simples (branch principal e branches de funcionalidade) com code review via *pull requests*. Um pipeline básico de CI/CD (por exemplo, GitHub Actions) será configurado para compilar o projeto, executar testes automatizados (testes de unidade e de integração dos serviços principais) e sinalizar falhas antes da integração na branch principal, reduzindo riscos de regressão. Sempre que possível, serão utilizadas ferramentas de qualidade de código (como linters e análise estática) e scripts de migração/carga de banco de dados, de forma que o processo de implementação produza não apenas o sistema em funcionamento, mas também um conjunto consistente de artefatos que documentem e respaldem a evolução do SGI.

Período (2026)	Atividades previstas
1ª quinzena de fevereiro	Revisar requisitos e Documento de Visão; revisar modelos (casos de uso, diagramas, modelo de dados).
2ª quinzena de fevereiro	Ajustar diagramas conforme feedback; detalhar backlog de histórias de usuário e priorizar funcionalidades.
1ª quinzena de março	Implementar infraestrutura do backend (projeto Spring Boot, banco de dados) e serviços básicos de autenticação.
2ª quinzena de março	Implementar funcionalidades de cadastro de clientes e regras básicas de negócio no backend.

1ª quinzena de abril	Implementar registro de inadimplências e atualização de saldo na API; criar scripts de carga de dados de teste.
2ª quinzena de abril	Desenvolver interfaces web/desktop (telas de login, clientes e inadimplências) e integrar com a API.
1ª quinzena de maio	Implementar funcionalidades de pagamentos, recálculo de saldo e relatórios principais (ex.: ranking de devedores).
2ª quinzena de maio	Realizar testes funcionais e de integração; corrigir defeitos; refinar usabilidade e mensagens da interface.
1ª quinzena de junho	Consolidar evidências de testes; revisar e atualizar o Documento de Projeto com base no sistema implementado.
2ª quinzena de junho	Finalizar Documento de Projeto, revisar normas e formatação; preparar e ensaiar apresentação.

Tabela 26- Cronograma

8. Processo de Implantação

Esta seção tem como finalidade apresentar o processo de implantação adotado durante o desenvolvimento do SGI — Sistema de Gerenciamento de Inadimplentes. O projeto foi organizado de forma incremental, seguindo ciclos de desenvolvimento com duração aproximada de 15 dias. Em cada ciclo, foram definidas atividades específicas, envolvendo revisão da documentação, ajustes nos modelos, implementação de funcionalidades, testes e preparação do ambiente de implantação.

Inicialmente, as primeiras etapas foram voltadas para a revisão dos requisitos, ajustes nos diagramas e organização do backlog de funcionalidades. Em seguida, foi iniciada a implementação da infraestrutura do backend, utilizando Spring Boot, Java 21 e banco de dados, além da criação dos serviços básicos de autenticação. Posteriormente, foram desenvolvidas as funcionalidades principais do sistema, como cadastro de clientes, registro de inadimplências, registro de pagamentos, atualização de saldo devedor, geração de relatórios e visualização de indicadores.

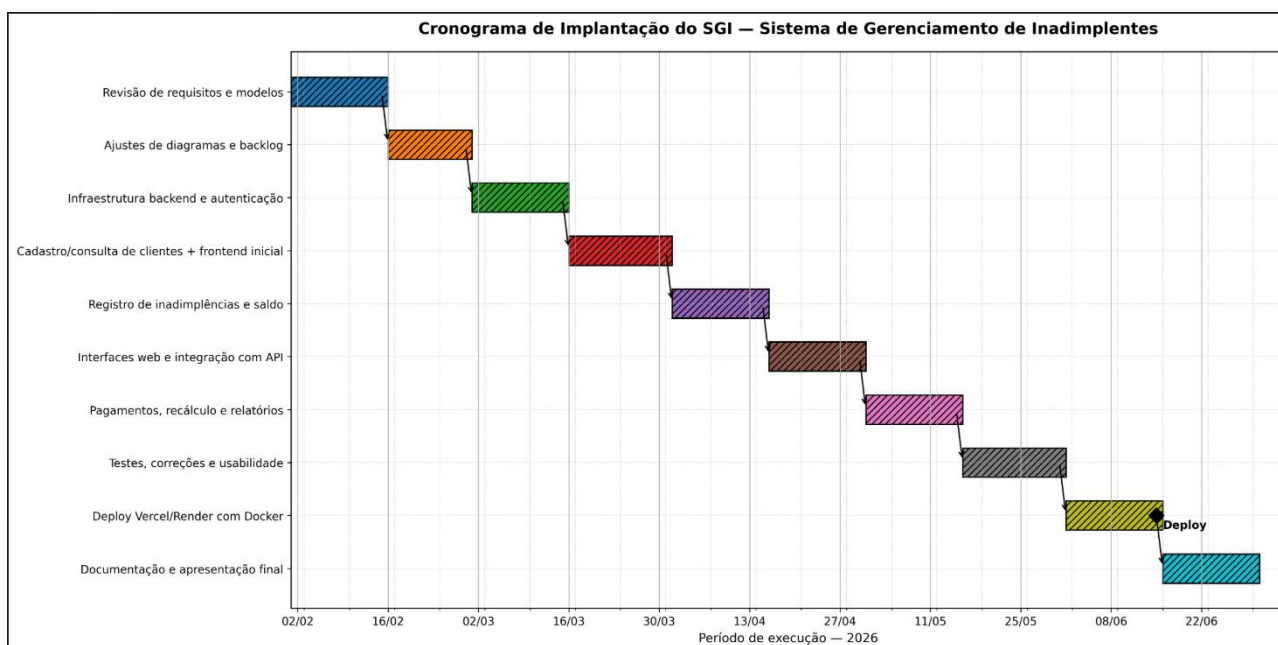


Figura 39- Cronograma

Como o sistema será disponibilizado em ambiente web, a implantação foi planejada considerando o frontend hospedado na Vercel e a API hospedada no Render. O frontend, desenvolvido com React + Vite, será responsável pela interface acessada pelo usuário por meio do navegador. Já o backend, desenvolvido com Spring Boot, será empacotado com Docker e implantado no Render, expondo os endpoints REST consumidos pelo frontend.

Durante o processo de implantação, também foram previstas configurações de ambiente, como variáveis de conexão com o banco de dados, regras de CORS, definição da URL da API no frontend e testes de comunicação entre as camadas. A camada de persistência será configurada em ambiente hospedado, preferencialmente utilizando um banco persistente, como PostgreSQL, garantindo maior segurança e preservação dos dados.

Após a implementação dos principais fluxos, foram realizadas etapas de testes funcionais e de integração, com o objetivo de validar o comportamento do sistema, corrigir defeitos encontrados e melhorar a usabilidade da interface. Ao final do processo, foram consolidadas as evidências de teste, a documentação técnica, os diagramas e o código-fonte do sistema, disponibilizados no repositório oficial do projeto no GitHub.

Dessa forma, o processo de implantação do SGI buscou garantir que a solução pudesse ser acessada pela web, sem necessidade de instalação local na máquina do cliente, mantendo uma arquitetura organizada, escalável e adequada ao escopo do trabalho.

9. Post-mortem

O desenvolvimento do software entregue proporcionou uma experiência importante de aprendizado sobre o processo de construção de um sistema real. Ao longo do projeto, foi possível perceber a importância de organizar bem as etapas de desenvolvimento, desde o levantamento de requisitos até a modelagem, implementação, testes e documentação da solução.

Durante o processo, alguns desafios surgiram, principalmente relacionados à definição do escopo, à organização das funcionalidades e à necessidade de adaptar a arquitetura do sistema conforme novas

decisões foram tomadas. Essas situações contribuíram para entender que o desenvolvimento de software é um processo iterativo, no qual ajustes e melhorias são naturais ao longo do tempo.

Também foi possível perceber a importância da comunicação com o cliente, pois as interações realizadas ajudaram a esclarecer necessidades, validar ideias e direcionar melhor as funcionalidades do sistema. Essa troca constante tornou o projeto mais alinhado ao problema real que se buscava resolver.

De forma geral, o post-mortem do projeto evidencia que a experiência foi positiva e enriquecedora. O desenvolvimento do SGI contribuiu para consolidar conhecimentos técnicos e práticos, além de reforçar a importância do planejamento, da documentação, da comunicação e da adaptação durante o processo de desenvolvimento de software.

9.1 Experiências Positivas

Durante o desenvolvimento do projeto, a experiência foi bastante positiva e contribuiu significativamente para o meu aprendizado. Foi interessante compreender melhor como um sistema é planejado, estruturado e desenvolvido, desde o levantamento de requisitos até a definição da arquitetura, modelagem dos dados, criação dos diagramas e organização das funcionalidades.

Outro ponto muito positivo foi a interação com o cliente, que ocorreu de forma frequente e colaborativa ao longo do projeto. Houve bastante troca entre as partes, o que facilitou o entendimento das necessidades reais do escritório e permitiu ajustar melhor as funcionalidades do sistema ao contexto de uso. Essa comunicação constante foi importante para aproximar o desenvolvimento da realidade do cliente e tornar a solução mais adequada ao problema identificado.

O projeto também permitiu ampliar minha visão sobre o funcionamento de sistemas em geral, principalmente em relação à separação de responsabilidades entre frontend, backend e banco de dados. Além disso, foi possível entender melhor a importância de documentar corretamente cada etapa do desenvolvimento, garantindo que as funcionalidades, regras de negócio e fluxos do sistema fiquem claros e bem organizados.

Além disso, a construção do SGI proporcionou aprendizado prático sobre modelagem de sistemas, casos de uso, diagramas UML, arquitetura em camadas, APIs REST e persistência de dados. Esses conhecimentos foram importantes para consolidar conteúdos vistos ao longo do curso e aproximar a teoria da prática.

De forma geral, o projeto foi uma oportunidade enriquecedora, pois possibilitou o desenvolvimento de uma solução voltada para um problema real, com participação ativa do cliente, contribuindo para minha evolução acadêmica e profissional na área de Engenharia de Software.

9.2 Experiências Negativas

No início do desenvolvimento do projeto, tive um pouco de dificuldade para organizar as ideias, definir melhor o escopo do sistema e entender como transformar as necessidades levantadas em funcionalidades reais. Esse processo exigiu ajustes, revisões e uma melhor compreensão sobre as etapas de desenvolvimento de software, principalmente em relação à modelagem, documentação e definição da arquitetura.

Outro ponto negativo foi a tentativa de realizar a conexão com a API do Sicoob, que inicialmente seria considerada para integração com recursos financeiros. No entanto, essa integração não foi

possível dentro do prazo do projeto, pois o sistema do banco apresentou instabilidades em alguns momentos, além de haver demora no retorno dos responsáveis para esclarecimento de dúvidas e apoio técnico. Esses fatores dificultaram o avanço da integração e fizeram com que essa funcionalidade não fosse incluída na versão entregue.

Apesar dessas dificuldades, os problemas enfrentados contribuíram para o aprendizado, pois mostraram a importância de lidar com limitações externas, planejar alternativas e adaptar o escopo do projeto conforme a viabilidade técnica e o tempo disponível.

9.3 Lições Aprendidas

Ao longo do desenvolvimento do projeto, uma das principais lições aprendidas foi a importância de definir bem o escopo do sistema desde as etapas iniciais. Durante o processo, foi possível perceber que quanto mais claras estão as funcionalidades, regras de negócio e limitações do projeto, mais fácil se torna organizar o desenvolvimento e evitar retrabalho.

Também foi possível aprender sobre a importância da comunicação com o cliente. As interações realizadas ajudaram a compreender melhor o problema real, validar decisões e ajustar o sistema conforme as necessidades identificadas. Essa experiência mostrou que o desenvolvimento de software não depende apenas da parte técnica, mas também de uma boa comunicação entre as partes envolvidas.

Outra lição importante foi entender que nem todas as funcionalidades planejadas inicialmente conseguem ser implementadas dentro do prazo ou das condições disponíveis. A tentativa de integração com a API do Sicoob, por exemplo, mostrou a necessidade de considerar riscos externos, como instabilidade de sistemas de terceiros e demora no suporte técnico. Com isso, ficou evidente a importância de planejar alternativas e priorizar as funcionalidades essenciais para a entrega.

Além disso, o projeto reforçou a importância da documentação, da modelagem e da organização da arquitetura. A criação dos diagramas, casos de uso, requisitos e modelos ajudou a visualizar melhor o funcionamento do sistema e facilitou o entendimento das responsabilidades de cada parte da aplicação.

De forma geral, o desenvolvimento do SGI contribuiu para consolidar conhecimentos técnicos e práticos em Engenharia de Software, principalmente sobre planejamento, arquitetura em camadas, APIs, persistência de dados, documentação e adaptação diante de mudanças no projeto.

9.4 Repositório do Trabalho

Todo o material desenvolvido ao longo do trabalho foi documentado e organizado no repositório oficial do projeto, disponível no GitHub por meio do seguinte link:

<https://github.com/ICEI-PUC-Minas-PPLES-TI/plf-es-2025-2-tcci-0393100-dev-matheus-pereira>

Nesse repositório encontram-se os artefatos produzidos durante o desenvolvimento do SGI — Sistema de Gerenciamento de Inadimplentes, incluindo a documentação do projeto, diagramas, modelos, códigos-fonte, registros de desenvolvimento e demais materiais relacionados ao trabalho.

10. TRABALHOS FUTUROS

Como trabalhos futuros, pretende-se evoluir o SGI — Sistema de Gerenciamento de Inadimplentes por meio da integração com a API do Sicoob. Essa funcionalidade permitirá que o sistema realize a emissão de boletos oficiais do banco diretamente pela aplicação, tornando o processo de cobrança mais automatizado e reduzindo a necessidade de emissão manual por outros canais.

Além da geração dos boletos, a integração também deverá possibilitar o recebimento das confirmações de pagamento dentro do próprio sistema. Dessa forma, quando um cliente realizar o pagamento de uma cobrança, a aplicação poderá registrar essa informação automaticamente, atualizar o saldo devedor da dívida e alterar seu status conforme a situação do pagamento.

Essa melhoria contribuirá para tornar o sistema mais completo e integrado ao processo financeiro do escritório, proporcionando maior agilidade, confiabilidade nas informações e redução de retrabalho. A conexão com a API do Sicoob também permitirá ampliar o controle sobre as cobranças, facilitando o acompanhamento dos boletos emitidos, pagos, vencidos ou cancelados.